GPU-Accelerated Finite-Volume Simulation of Non-Newtonian Blood Flow in the Left Anterior Descending Artery

Staggered-Grid Projection Method with QUICK Convection, Bird–Carreau Rheology and Block-Corrected SOR in CUDA C

Course Project 1 – OE5450 Numerical Techniques in Ocean Hydrodynamics (Final Project, Jan–May 2025)

Karthick J (OE23S018)
Department of Ocean Engineering, IIT Madras

Revised report, October 2026

Abstract

Steady laminar flow of blood, modelled as a shear-thinning Bird–Carreau fluid, is computed in a two-dimensional model of the proximal left anterior descending (LAD) coronary artery (length 140 mm, width 3.5 mm, mean velocity 0.39 m/s) with a finite-volume projection method on a staggered grid of 1400×35 cells. Every operation of the time step (boundary conditions, QUICK/central momentum predictor, divergence, pressure-correction sweeps, velocity correction, shear-rate and viscosity update, reductions) is a CUDA kernel. The submitted code did not compile; once it compiled, it produced NaN in the first time step. Twenty-five defects were corrected without changing the structure of the code; the most important were a pressure equation with the wrong sign that applied the pressure gradient twice, an outlet that was blocked because the last velocity face was never updated, walls moved one cell inwards, viscosity interpolated half a cell off, out-of-bounds QUICK stencils, an explicit time step 3.8 times the stability limit, and a pressure solver that cannot conserve mass on a 400:1 domain. The last point was fixed with a block (additive) correction along the artery, which reduced the mass error after the start-up from 4% to 10^{-7} and the pressure-solver work four-fold. The kernels were verified by serial CPU emulation. In Newtonian mode the solver reproduces the exact solution of its discrete equations to 2.6×10^{-6} and the Poiseuille pressure gradient to 0.12%; with Bird–Carreau rheology the developed velocity, shear-rate and viscosity profiles agree with an independent ODE solution within 0.2%, and the pressure gradient within 0.07%. Shear-thinning lowers the pressure drop over the 140 mm segment from 2994 Pa (Newtonian, μ_0) to 335 Pa, flattens the core, and gives a wall shear stress of 4.2 Pa and a development length of 50 mm ($14W$).

Contents

1 Problem definition **3**

1.1 Physiological data . . . 3

1.2 Governing equations and boundary conditions . . . 3

2 Numerical method **3**

2.1 Staggered finite-volume grid . . . 3

2.2 Momentum predictor . . . 4

2.3 Pressure correction (projection) . . . 4

2.4 Explicit stability limit . . . 5

2.5 CUDA implementation 5
2.6 Algorithm 6

3 Code review of the submitted program **7**

4 Verification: Newtonian Poiseuille flow **8**

5 Results: Bird–Carreau flow in the LAD **8**
5.1 Convergence and mass conservation 9
5.2 Flow field 9
5.3 Developed flow: comparison with the exact Carreau solution 10
5.4 Pressure and wall shear stress along the artery 11
5.5 Block correction: effect on mass conservation and cost 12
5.6 Run time 12

6 Limitations **12**

7 Conclusions **12**

A Source code **13**

1 Problem definition

1.1 Physiological data

The proximal LAD segment is represented, as in the original report, by a straight two-dimensional channel (Table 1). Blood is treated as an incompressible generalised Newtonian fluid of density ρ whose apparent viscosity follows the Bird–Carreau law

$$\mu(\dot\gamma) = \mu_\infty + (\mu_0 - \mu_\infty)\big[1 + (\lambda\dot\gamma)^a\big]^{(n-1)/a}, \qquad \dot\gamma = \sqrt{2\Big(\frac{\partial u}{\partial x}\Big)^2 + 2\Big(\frac{\partial v}{\partial y}\Big)^2 + \Big(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}\Big)^2}, \tag{1}$$

where $\dot\gamma$ is the shear rate (second invariant of the rate-of-strain tensor).

Quantity	Symbol	Value	Note
length, width	L, W	0.14 m, 3.5 mm	proximal LAD (Ali et al. 2024)
mean inflow velocity	U	0.39 m/s	time-averaged; inlet profile $1.5U(1-\eta^2)$
density	ρ	1060 kg/m^3	
Carreau parameters	$\mu_0, \mu_\infty, \lambda, n, a$	0.056 Pa s, 0.0035 Pa s, 3.13 s, 0.6, 2	original report†
grid	$N_x \times N_y$	1400×35, $\Delta x = \Delta y = 0.1$ mm	49 000 cells
Reynolds number	$\rho U W/\mu$	25.8 (μ_0), 245 (μ_w), 413 (μ_∞)	μ_w: developed wall value

Table 1: Parameters. †The widely used fit of Cho & Kensey (1991) has $\mu_\infty = 0.00345$ Pa s, $\lambda = 3.313$ s, $n = 0.3568$; the values of the original project are kept here.

1.2 Governing equations and boundary conditions

$$\nabla \cdot \mathbf{u} = 0, \tag{2}$$

$$\rho\Big(\frac{\partial \mathbf{u}}{\partial t} + \nabla\cdot(\mathbf{u}\mathbf{u})\Big) = -\nabla p + \nabla\cdot\big(\mu(\dot\gamma)\nabla\mathbf{u}\big). \tag{3}$$

The viscous term is used in the Laplacian form of the original code. For variable viscosity the full stress divergence contains in addition $\nabla\cdot(\mu\nabla\mathbf{u}^T) = (\nabla\mu)\cdot(\nabla\mathbf{u})^T$; this term vanishes identically in fully developed flow ($v = 0$, $\partial u/\partial x = 0$) and is small in the gently developing entrance flow considered here, so it was kept as a documented modelling simplification rather than changed.

Boundary conditions: parabolic inflow $u = 1.5U(1-\eta^2)$, $v = 0$ at $x = 0$ ($\eta = 2y/W - 1$); no slip at $y = 0, W$; at $x = L$ zero streamwise gradient of u, $v = 0$ in the last cell column and fixed pressure $p = 0$. The fluid is initially at rest with a linear pressure field $p = \Delta P(1 - x/L)$, $\Delta P = 2886.3$ Pa (the original Poiseuille estimate); the steady state is reached by time marching.

2 Numerical method

2.1 Staggered finite-volume grid

Pressure, viscosity and shear rate are stored at cell centres (i, j); $u_{i,j}$ on the west face of cell i ($i = 0 \ldots N_x$, index `i+j*(nx+1)`) and $v_{i,j}$ on the south face of cell j ($j = 0 \ldots N_y$, index `i+j*nx`). Each velocity component has its own control volume centred on its face (Fig. 1). No-slip on the walls is imposed through antisymmetric ghost values, $u_{i,-1} = -u_{i,0}$, $u_{i,N_y} = -u_{i,N_y-1}$, so that the wall lies exactly on $y = 0$ and $y = W$.

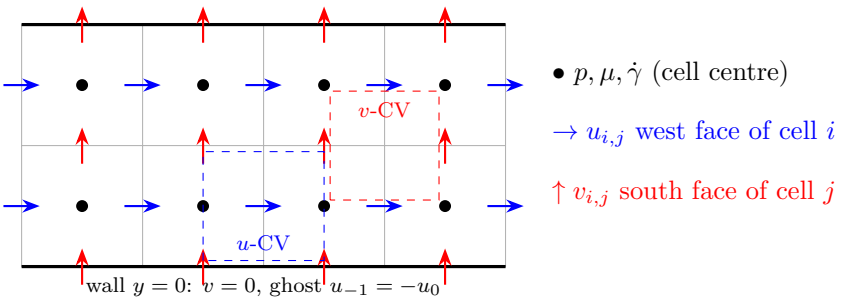


Figure 1: MAC arrangement near the lower wall and the two momentum control volumes. Faces of the u-CV lie at cell centres (east/west) and at cell corners (north/south); this determines where viscosity and transverse velocity must be interpolated.

2.2 Momentum predictor

Explicit Euler with the pressure of the previous step gives the intermediate velocity, e.g. for the u-CV of face (i, j):

$$u^*_{ij} = u^n_{ij} + \frac{\Delta t}{\Delta x \Delta y}\Big[-(F_e\phi_e - F_w\phi_w + F_n\phi_n - F_s\phi_s) + \frac{D_e(u_E - u_P) - D_w(u_P - u_W) + D_n(u_N - u_P) - D_s(u_P - u_S)}{\rho}\Big] - \frac{\Delta t}{\rho}\frac{p_{ij} - p_{i-1,j}}{\Delta x}, \quad (4)$$

with volume fluxes $F_e = \frac{1}{2}(u_P + u_E)\Delta y$, $F_n = \frac{1}{2}(v_{i-1,j+1} + v_{i,j+1})\Delta x$ (two-point averages on the CV faces), diffusion conductances $D_e = \mu_e \Delta y/\Delta x$, $D_n = \mu_n \Delta x/\Delta y$ with $\mu_e = \mu_{i,j}$, $\mu_w = \mu_{i-1,j}$ (cell centres) and μ_n, μ_s the four-cell averages at the corners. Face values ϕ_f use QUICK (Leonard 1979),

$$\phi_f = \tfrac{6}{8}\phi_U + \tfrac{3}{8}\phi_D - \tfrac{1}{8}\phi_{UU}, \quad (5)$$

(U upstream, D downstream, UU far upstream of the face, chosen by the sign of F_f), and first-order upwind where UU would lie outside the domain (first interior faces). The v-equation is analogous.

2.3 Pressure correction (projection)

Writing $\mathbf{u}^{n+1} = \mathbf{u}^* - (\Delta t/\rho)\nabla p'$ and requiring $\nabla \cdot \mathbf{u}^{n+1} = 0$ gives, cell by cell,

$$\sum_{nb} a_{nb}\,(p'_{nb} - p'_P) = \frac{\rho}{\Delta t}\,(\nabla \cdot \mathbf{u}^*)_P, \qquad a_{E,W} = \frac{1}{\Delta x^2},\ a_{N,S} = \frac{1}{\Delta y^2}, \quad (6)$$

with $a_{nb} = 0$ across faces whose velocity is not corrected (inlet face, walls, v-faces of the inlet column), i.e. $\partial p'/\partial n = 0$ there, and $p' = 0$ in the outlet column. Then $u^{n+1} = u^* - (\Delta t/\rho)\,\delta_x p'$, $v^{n+1} = v^* - (\Delta t/\rho)\,\delta_y p'$ and $p^{n+1} = p^n + p'$ (incremental projection). Because the predictor already contains ∇p^n, the zero initial guess for p' is equivalent to a warm start $p^{n+1} = p^n$, and $p' \to 0$ at steady state, where the converged solution satisfies the discrete momentum and continuity equations exactly. Eq. (6) is solved by red–black SOR ($\omega = 1.85$) until the mean change per sweep is below 10^{-6} Pa.

Why point SOR alone fails on an artery-shaped domain. The slowest error mode of Eq. (6) is uniform across the channel and varies over the whole length (Neumann inlet, Dirichlet outlet): wavenumber $\theta_x = \pi/(2N_x)$, Jacobi spectral radius $\rho_J = (1 + \cos\theta_x)/2 \approx 1 - \theta_x^2/4 = 1 - 3.1 \times 10^{-7}$. For SOR with ω below the optimum of this mode the eigenvalue is $1 - \delta$ with $\delta = 2\omega\epsilon/(2 - \omega)$, $\epsilon = 1 - \rho_J$: $\delta = 6\epsilon = 1.9 \times 10^{-6}$ per sweep for the original $\omega = 1.5$. Even 1000 sweeps per step therefore reduce this mode by only 0.2%. Physically it is the cross-sectional flow rate $Q(x)$: without a converged correction, mass is not conserved along the artery (Section 5.5).

Block correction. Following Settari & Aziz (1973) and Patankar (1980), every ten sweeps a constant c_i is added to all cells of column i such that the column-summed residual $R_i = \sum_j r_{ij}$ of Eq. (6) vanishes. Summing Eq. (6) over a column, the north/south terms cancel and

$$A_W\, c_{i-1} - (A_W + A_E)\, c_i + A_E\, c_{i+1} = R_i, \qquad A_E = A_W = \frac{N_y}{\Delta x^2}\ (A_W = 0 \text{ at the inlet}), \quad c_{N_x-1} = 0, \tag{7}$$

a tridiagonal system solved by the Thomas algorithm. For this separable operator the correction removes the cross-sectionally uniform component of the error exactly, leaving modes with $\theta_y \geq \pi/N_y$, for which $\rho_J \leq 0.998$ and $\omega_{opt} = 2/(1 + \sin(\pi/N_y)) \approx 1.84$; this is why $\omega = 1.85$ was chosen. The result is that every cross-section carries exactly the inflow, and the solver converges in 17–25 sweeps per step.

2.4 Explicit stability limit

For explicit Euler with central/QUICK fluxes, Fourier analysis of the linearised predictor gives two conditions. For the highest wavenumber ($\theta = \pi$ in both directions) the amplification factor is $G = 1 - 8d - \sigma_Q$ with $d = \nu\Delta t/\Delta x^2$ and σ_Q the QUICK (upwind-biased) contribution $\approx$ CFL; $|G| \leq 1$ requires

$$\Delta t \leq \Big[\frac{|u|}{\Delta x} + \frac{|v|}{\Delta y} + \frac{2\mu}{\rho}\Big(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}\Big)\Big]^{-1}. \tag{8}$$

For low wavenumbers the forward-Euler central convection must be damped by diffusion, $|G|^2 \approx 1 + (\sigma^2 - 2d)\theta^2$, which requires

$$\Delta t \leq \frac{2\mu}{\rho|\mathbf{u}|^2}. \tag{9}$$

Both are evaluated cell by cell (with the local μ) by a reduction kernel and applied with a safety factor 0.9, together with CFL ≤ 0.5 and a growth limit of 1.5 per step. With $\mu = \mu_0$ on the centreline Eq. (8) gives $\Delta t = 3.34 \times 10^{-5}$ s. The original step, $0.5\Delta x/U = 1.28 \times 10^{-4}$ s, is 3.8 times larger. Even the pure diffusion limit $\rho\Delta x^2/(4\mu_0) = 4.7 \times 10^{-5}$ s is not sufficient: a run at that value developed a sustained cell-to-cell oscillation at the outlet with velocities of 1.1 m/s.

2.5 CUDA implementation

Kernel	threads	work per thread
`ApplyBoundaryConditions`	$(N_x+1)(N_y+1)$	inlet profile, outlet $\partial u/\partial x = 0$, p_{out}, wall $v = 0$
`Xmomentum_solver`	one per u-face	QUICK fluxes, face viscosities, predictor u^*
`YMomentum_solver`	one per v-face	same for v^*
`Divergence`	one per cell	$(\nabla \cdot \mathbf{u}^*)_P$
`PoissonSolverGS` (2 launches)	one per cell, red / black	SOR update of p'; `atomicAdd` of $\lvert\delta p'\rvert$
`ColumnResidual`	one per column	$R_i = \sum_j r_{ij}$ (35 cells)
`BlockCorrectionTDMA`	1	Thomas algorithm, 1399 unknowns
`ApplyBlockCorrection`	one per cell	p'_{ij} += c_i
`CorrectVelocities`	one per face/cell	projection, p += p'
`Shear_rate`	one per cell	$\dot\gamma$ with ghost values, Bird–Carreau μ
`CalculateResidual`, `CalculateMaxVelocity`	one per face/cell	`atomicAdd`/`atomicMax` reductions for convergence and Δt

Table 2: Kernels (16×16 thread blocks; the launch grid covers $(N_x+1) \times (N_y+1)$ so that the outlet and top-wall faces have threads).

All kernels except the SOR sweep write to arrays they do not read (Jacobi-style), and the SOR sweep is split into red and black half-sweeps, so there are no data races. Float reductions use

`atomicMax` on the integer image of non-negative floats, which preserves ordering. Each array holds ≈0.2 MB, so the whole state (≈3 MB) fits in the L2 cache of a modern GPU; the cost per step is dominated by kernel-launch and synchronisation overhead of the 2 launches and one host copy per SOR sweep.

Verification without a GPU. The workspace used for this revision has no GPU. The kernels were executed by `tools/cuda_cpu_shim.h`: `tools/emulate.sh` rewrites every launch `k<<<G,B>>>(args)` into a loop over `blockIdx`/`threadIdx` that calls the unmodified kernel once per thread, and maps the runtime API (`cudaMalloc`, `cudaMemcpy`, `cudaMemset`, events, `atomicAdd`, `atomicMax`, `__float_as_int`) to host equivalents. Because no kernel has a data race, the serial execution reproduces the GPU result up to the summation order of the atomic adds. `run.sh` compiles the same file with `nvcc` and repeats the runs on the GPU when one is present.

2.6 Algorithm

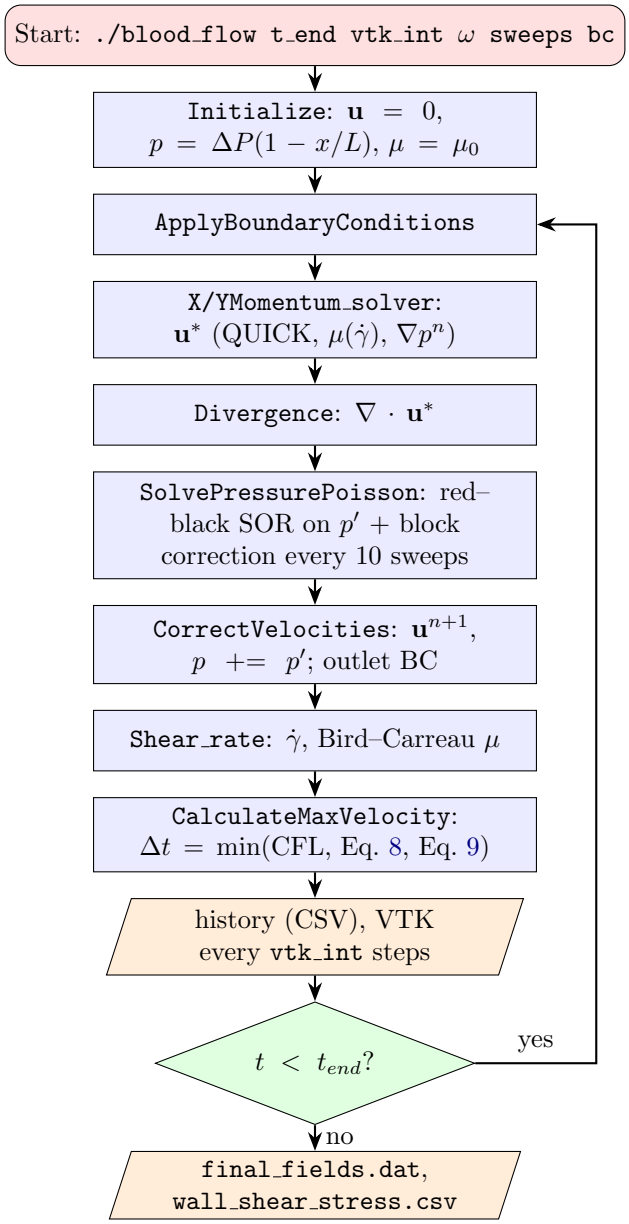


Figure 2: Time step of `code/Blood_flow.cu`; all boxes except I/O are CUDA kernels.

3 Code review of the submitted program

The submitted file is kept as `original/Blood_flow_original.cu` (the CPU port and the listing printed in the report are kept alongside). It does not compile: kernel parameters named `dt`, `DX`, `DY`, `rho` are replaced by the macros of the same name, and two prototypes contain `int 1401, int 36`. With only these compile errors removed (`original/Blood_flow_original_compile_fixes_only.cu`) the program runs but the residual is NaN from the first step and the pressure is NaN in 47 600 of the 49 000 cells after 1.3 ms (`original/run_of_original_compile_fixed_only_log.txt`). The original report contains no simulation results, so there are no earlier plots to compare with. The errors and corrections (marked `[FIX]` in `code/Blood_flow.cu`) are:

Location	Error → correction
prototypes, kernel parameters	Macro/parameter name clashes and numeric parameter names: did not compile → renamed `dt_val`, `dx`, `dy`, `rho_val`.
`NX`, `NY`	`(int)(L/DX)+1` counts nodes, but the arrays hold cells: 1401×36 cells = 140.1×3.6 mm → rounded cell counts 1400×35.
inlet profile	$\eta = 2j/(N_y - 1) - 1$ puts zero velocity in the first and last fluid rows (cell centres), and $U(1-\eta^2)$ has mean $2U/3 = 0.26$ m/s instead of the stated mean 0.39 m/s → $\eta = 2(j+\frac{1}{2})/N_y - 1$, $1.5U(1-\eta^2)$.
inlet pressure	$p_0 = p_1$ imposed every step removes the pressure force on face 1 → removed; inlet condition enters the pressure-correction equation.
wall rows of u	$u = 0$ imposed on rows $j = 0$ and $N_y - 1$, which lie inside the fluid: walls moved one cell inwards (33 fluid rows) → ghost-cell no-slip.
momentum ranges	u solved for $1 \le i < N_x - 1$, $1 \le j < N_y - 1$: face $N_x - 1$ never updated, so the outlet face kept $u = 0$ and *blocked the channel*; wall rows frozen. v solved for $j < N_y - 1$ → all interior faces.
uninitialised $\mathbf{u}^*$	boundary faces of u^*, v^* never written (raw `cudaMalloc` memory) but read by `Divergence` and `CorrectVelocities` → copied from $\mathbf{u}^n$.
QUICK stencil	reads u_{i-2} at $i = 1$ and $u_{j\pm2}$ next to the walls: out-of-bounds memory → upwind where the far-upstream node does not exist.
transverse velocity on CV faces	four-point averages give v (resp. u) at the CV centre and were used for both faces → two-point face averages.
face viscosities	averaged with the wrong neighbours (half a cell off) → cell values on east/west faces, four-cell corner averages on north/south faces.
shear rate	cross derivatives were one-sided (at $y_{j+1/2}$); wall-adjacent and inlet/outlet cells were given $\mu = \mu_0$, the *largest* viscosity where the shear rate is largest → central differences with ghost values in all cells.
divergence	computed for interior cells only; boundary-cell entries uninitialised → all cells.
pressure equation sign	source $-\rho\nabla\cdot\mathbf{u}^*/\Delta t$ has the wrong sign for $\mathbf{u} = \mathbf{u}^* - (\Delta t/\rho)\nabla p'$ → $+$.
pressure equation Δt	used the macro $\Delta t = 10^{-5}$ s, not the adaptive step taken → actual Δt.
pressure unknown	solved for the full p although the predictor already contains ∇p, then corrected with ∇p again (pressure counted twice) → incremental correction p', $p \mathrel{+}= p'$.
SOR race	all cells updated in one launch while reading neighbours from the array being written (non-deterministic Jacobi/Gauss–Seidel mix) → red–black ordering.
pressure BCs	boundary cells never updated in the solver → Neumann (walls, inlet) / Dirichlet (outlet) built into the coefficients.
mass conservation	point SOR cannot correct the axial flow rate on a 1400×35 domain (Section 2.3) → block correction.
"SIMPLE" loop	each of up to 1000 inner passes re-predicted from the current $\mathbf{u}$ with the full Δt, i.e. advanced the flow by another step while the clock advanced once → one predictor–projection per step (explicit fractional-step method).
time step	CFL step 1.28×10^{-4} s is 3.8 times the stability limit → Eqs. (8)–(9); growth limiter `fminf(dt,1.5*dt)` was a no-op → limits growth relative to the previous step; final step split to avoid a $\sim$ns step.
residual kernel	loop bounds missed the outlet faces and the top wall row → all faces; reported as RMS $\partial u/\partial t$.

Location	Error → correction
memory management	`cudaMalloc/cudaFree` inside the time loop and inside every pressure solve → allocated once.
launch grid	sized by N_x, N_y, covering the N_x+1 / N_y+1 faces only by accident → $(N_x+1)\times(N_y+1)$.
VTK output	`DIMENSIONS` counts points; writing $N_x \times N_y$ points dropped the last cell row and column → $(N_x+1)\times(N_y+1)$; viscosity and shear rate added.
input	interactive `scanf` → command-line arguments (batch runs).

4 Verification: Newtonian Poiseuille flow

With $\lambda = 0$ the Carreau law reduces to a Newtonian fluid with $\mu = \mu_0$. For this case the exact solution of the *discrete* equations is known: substituting $u_j = \frac{G}{2\mu} y_j (W - y_j) + c$ into the five-point operator with the ghost condition $u_{-1} = -u_0$ gives $c = Gh^2/(8\mu)$, and the flow-rate constraint then gives $G_{disc} = 12\mu\bar{U}_d/[W^2(1+2/N_y^2)]$, where $\bar{U}_d = 1.000408\,U$ is the midpoint-rule mean of the sampled inlet parabola. A 20 mm channel was run to $t = 0.3$ s (13 viscous time constants $W^2/\pi^2\nu$).

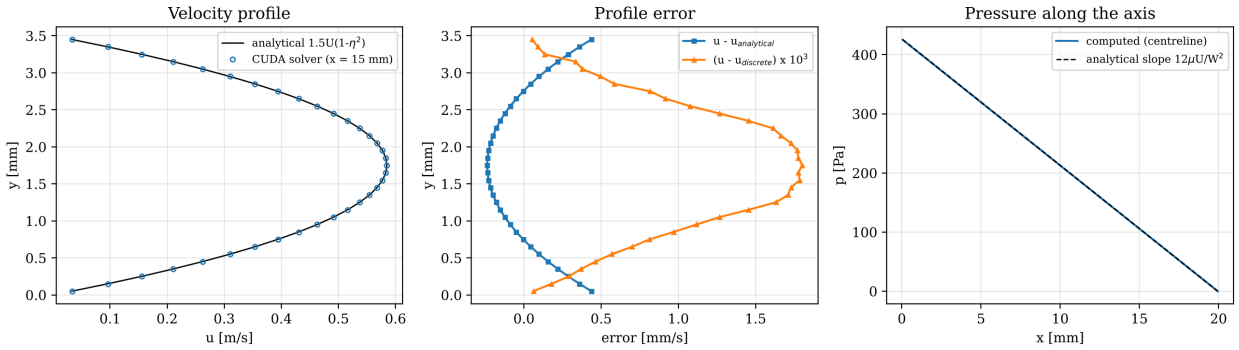


Figure 3: Newtonian verification at $x = 15$ mm: profile, error relative to the analytical and to the exact discrete solution, and axial pressure.

Quantity	computed	analytical (rel. error)	exact discrete (rel. error)
$G = -dp/dx$ [Pa/m]	21 368.08	21 394.29 (-1.2×10^{-3})	21 368.13 (-2.6×10^{-6})
$\max\lvert u - u_{ref}\rvert$ [m/s]	–	4.4×10^{-4} (0.075% of u_{max})	1.8×10^{-6}
τ_w [Pa]	37.394	37.440 (-1.2×10^{-3})	–

Table 4: Newtonian verification.

The solver reproduces its own exact discrete solution to single-precision round-off, which verifies the momentum predictor, the projection, the pressure boundary conditions and the ghost-cell walls; the remaining difference from the analytical solution is the expected $O(h^2)$ discretisation error ($2/N_y^2 = 0.16\%$ in G).

5 Results: Bird–Carreau flow in the LAD

The full domain was integrated to $t = 2$ s (59 961 steps, 5.6 flow-through times L/U, 9 viscous time constants based on μ_w).

5.1 Convergence and mass conservation

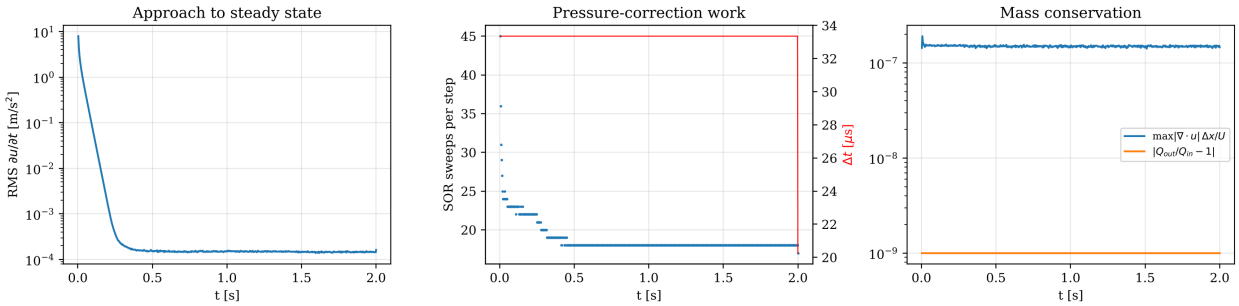


Figure 4: RMS $\partial u/\partial t$, SOR sweeps per step and time step, and mass-conservation errors.

The RMS rate of change falls by five orders of magnitude within 0.3 s and then stays at 1.5×10^{-4} m/s^2, which corresponds to a change of 5×10^{-9} m/s per step, the single-precision round-off level. The pressure solver needs 45 sweeps per step at start-up and 18 at steady state. The divergence after projection is $\leq 1.9 \times 10^{-7} U/\Delta x$ and the outflow equals the inflow to round-off throughout.

5.2 Flow field

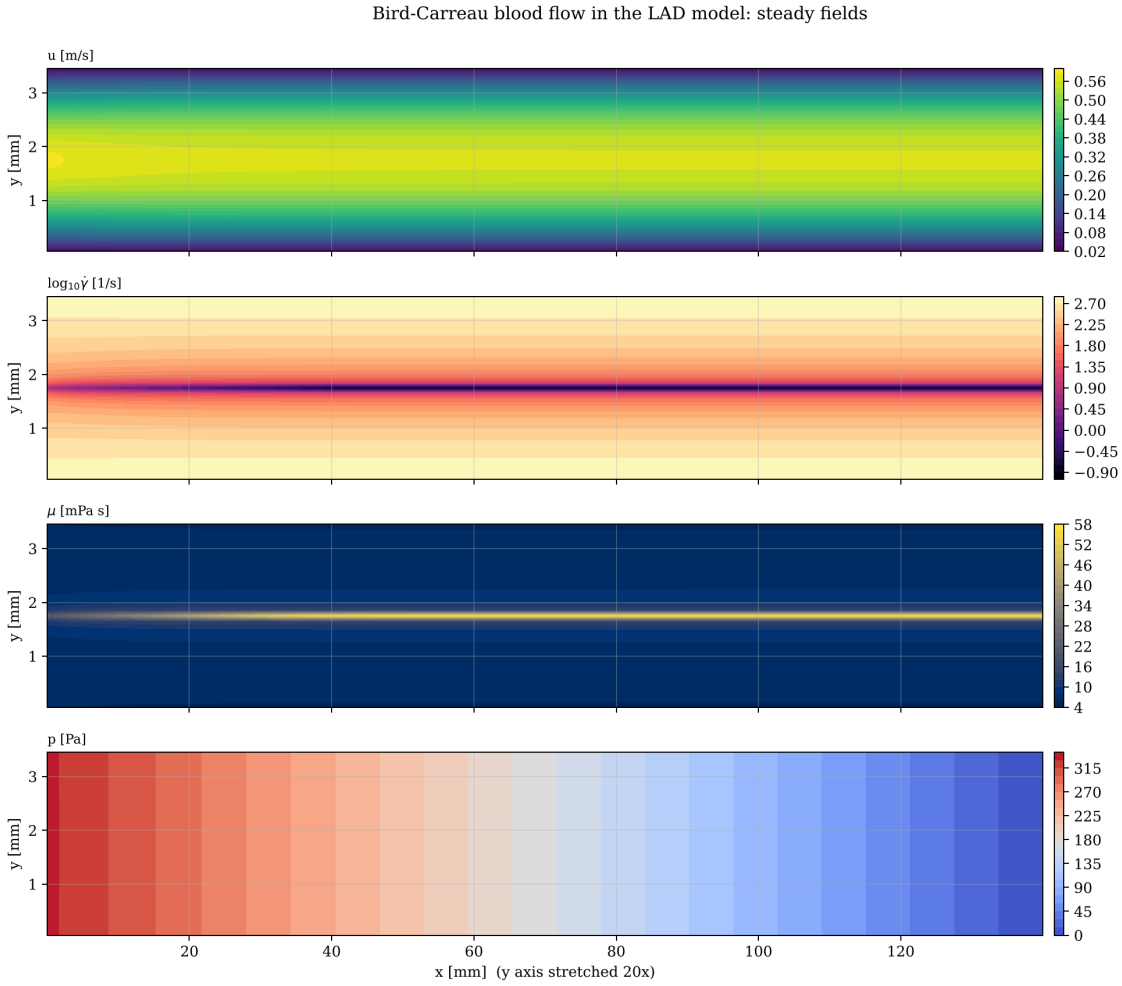


Figure 5: Steady u, shear rate, viscosity and pressure (the y axis is stretched 20 times).

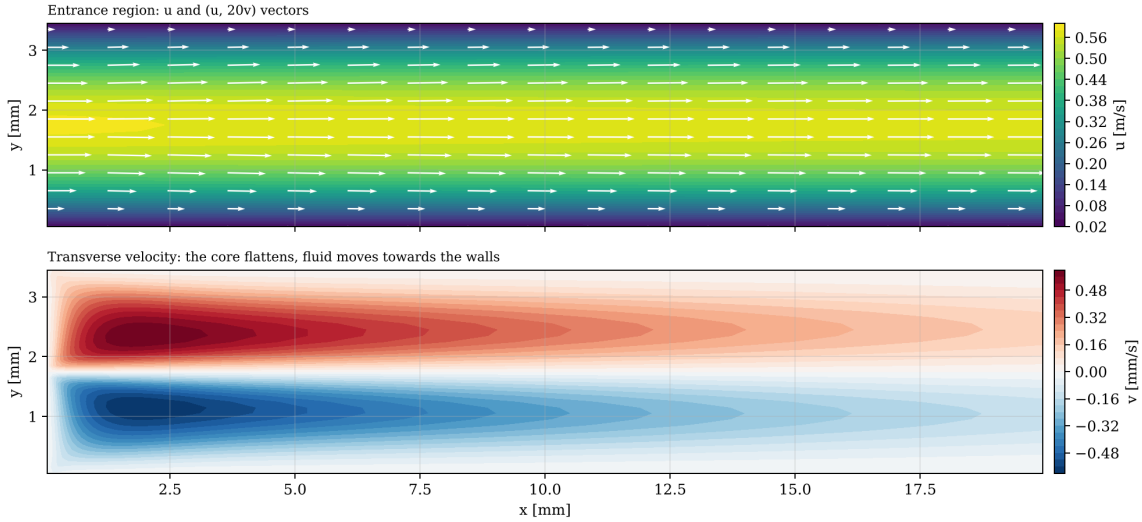


Figure 6: Entrance region: the parabolic inflow is blunted by shear-thinning; the transverse velocity (up to 0.6 mm/s) carries fluid from the core towards the walls.

Shear-thinning concentrates the viscosity in a thin core around the centreline, where $\dot{\gamma} \to 0$ and $\mu \to \mu_0 = 56$ mPa s, while over most of the section $\dot{\gamma} = 10^2$–7×10^2 s^{-1} and $\mu = 6$–10 mPa s. The parabolic inflow (the Newtonian developed profile) is therefore not an equilibrium profile: the core decelerates, the near-wall fluid accelerates, and the transverse velocity of Fig. 2 redistributes the flow until the developed Carreau profile is reached.

5.3 Developed flow: comparison with the exact Carreau solution

For fully developed flow, $d(\mu\dot{\gamma})/ds = -G$ gives $\mu(\dot{\gamma})\dot{\gamma} = Gs$ (s = distance from the centreline), which is inverted for $\dot{\gamma}(s)$ (monotone because $n > 0$) and integrated, $u(s) = \int_s^{W/2} \dot{\gamma}\, ds'$; G follows from $\frac{2}{W}\int_0^{W/2} u\, ds = \bar{U}_d$ (bisection, 4001 points, `code/postprocess.py`).

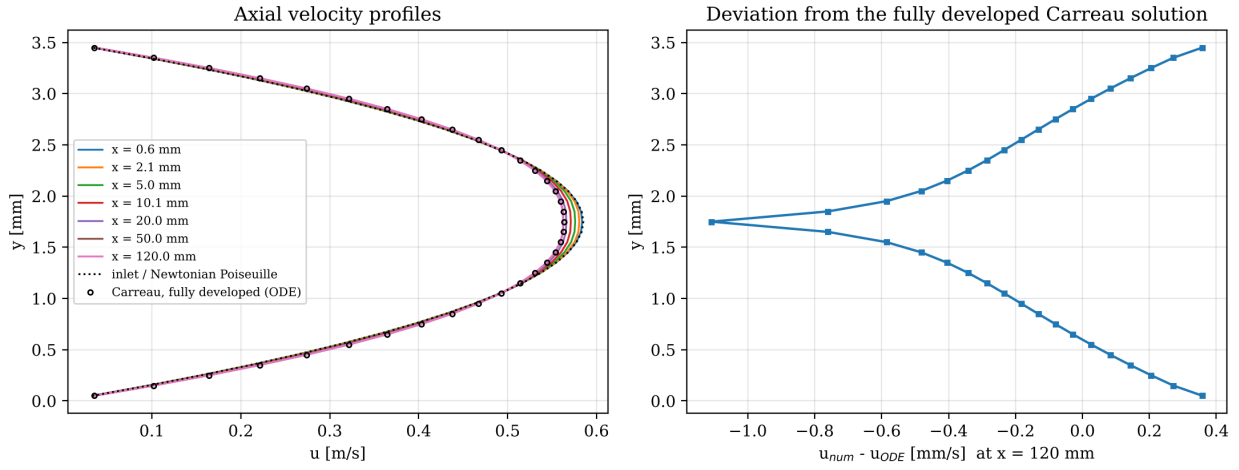


Figure 7: Velocity profiles along the artery and deviation from the exact developed Carreau profile at $x = 120$ mm.

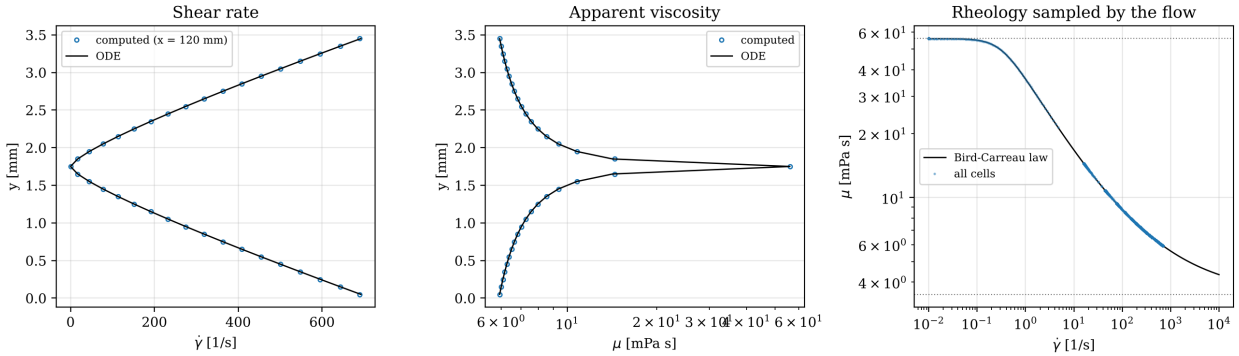


Figure 8: Shear rate and viscosity across the developed section against the ODE, and the part of the Carreau curve sampled by the flow.

Quantity	computed	exact developed (rel. error)	Newtonian μ_0 / μ_∞
$G = -dp/dx$ [Pa/m] ($x > 80$ mm)	2414.4	2416.2 (-7.4×10^{-4})	21 403 / 1338
centreline velocity [m/s]	0.5625	0.5636 (-0.20%)	0.585 / 0.585
$\max \lvert u - u_{ODE} \rvert$ [m/s]	1.1×10^{-3}	(0.20% of u_c)	–
wall shear stress τ_w [Pa]	4.204	4.228 (-0.59%)	37.46 / 2.34
wall shear rate [s^{-1}], wall viscosity [mPa s]	–	717, 5.90	669, 56 / 3.5
pressure drop over the segment [Pa]	334.7	338.0 ($G\,(L - \Delta x)$)	2994 / 187
development length L_e (1% criterion)	50.4 mm = $14.4W$	–	–

Table 5: Developed-flow quantities. The largest local error (0.2%) is in the centreline cell, where μ varies by a factor of five within one cell.

5.4 Pressure and wall shear stress along the artery

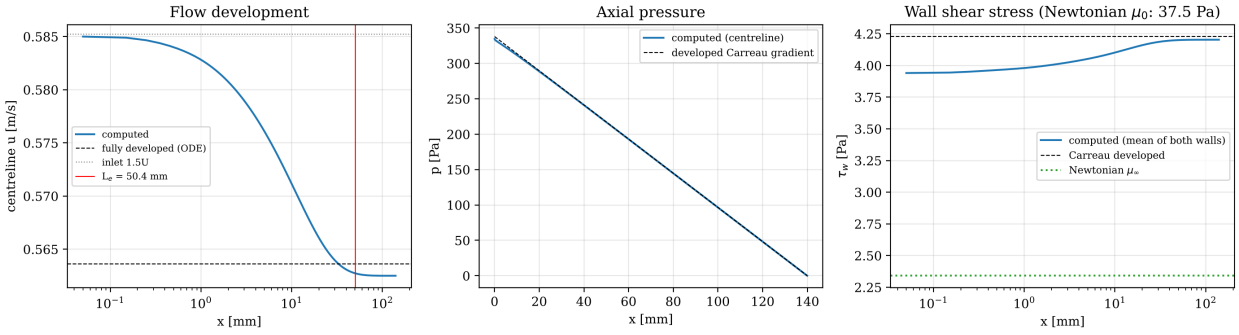


Figure 9: Centreline velocity, axial pressure and wall shear stress.

The pressure falls linearly after the entrance region, with the developed Carreau gradient. The wall shear stress rises from 3.94 Pa at the inlet (parabolic profile with μ evaluated at the Newtonian wall shear rate) to the developed value 4.20 Pa as the profile blunts and the wall shear rate increases from 669 to 717 s^{-1}. A Newtonian model with the zero-shear viscosity would overpredict the pressure drop and the wall shear stress by a factor of nine; the high-shear-limit model underpredicts them by 45%. Since WSS in the range 1–7 Pa is the physiological reference for coronary arteries, the choice of rheological model is not a detail.

5.5 Block correction: effect on mass conservation and cost

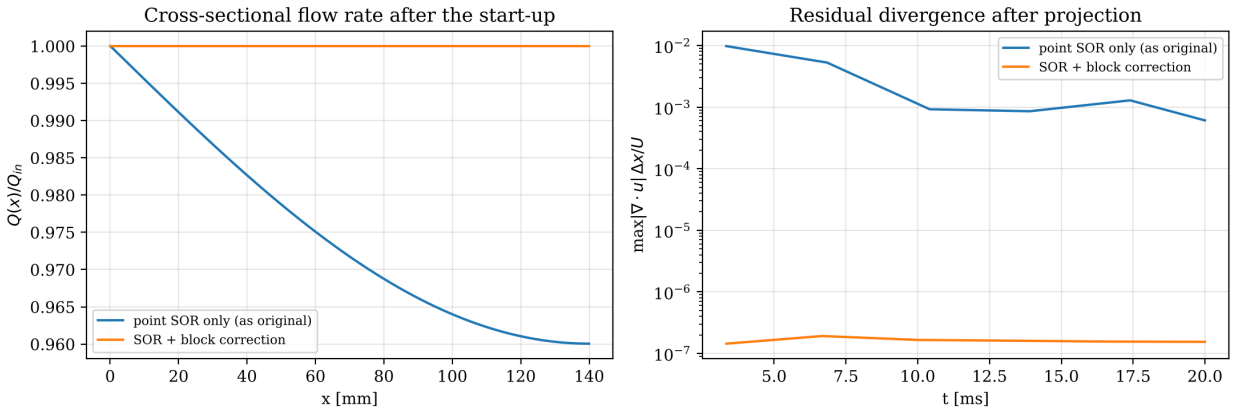


Figure 10: First 20 ms of the full case with point SOR only (1000 sweeps per step, as the original) and with the block correction.

With point SOR every step uses the maximum of 1000 sweeps, the residual divergence stays at $6 \times 10^{-4} U/\Delta x$, and after 20 ms only 96% of the inflow leaves the artery. With the block correction each cross-section carries the inflow to round-off, the solver converges in about 25 sweeps, and the run is 4.3 times faster (58.6 against 252.9 ms per step in the emulation).

5.6 Run time

The complete $t = 2$ s run takes 612 s in the serial CPU emulation (10.2 ms per step, 49 000 cells, about 18 SOR sweeps per step), the Newtonian check 24 s. On a GPU the cost per step is dominated by the ≈50 kernel launches and the per-sweep host copy of the convergence residual; `run.sh` records the GPU times in `results/timing.txt`. Checking the residual every few sweeps, fusing the red and black half-sweeps into one launch with shared-memory tiles, and moving the reductions to warp-level primitives are the natural next optimisations.

6 Limitations

- The artery is a straight two-dimensional channel (planar Poiseuille base flow), not an axisymmetric or patient-specific tube; curvature, branches, stenoses and wall motion are not modelled.
- The inflow is steady (time-averaged velocity); coronary flow is pulsatile, with a Womersley number of order 2–3.
- The viscous term uses the Laplacian form $\nabla\cdot(\mu\nabla\mathbf{u})$ of the original code; the omitted $\nabla\cdot(\mu\nabla\mathbf{u}^T)$ vanishes in developed flow.
- Single precision limits the steady residual to $\sim 10^{-4}$ m/s^2, which is adequate for the quantities reported.

7 Conclusions

- The submitted CUDA finite-volume code did not compile, and with compile errors removed it produced NaN in the first step. Twenty-five defects (listed in Section 3) were corrected while

keeping the kernels and the data layout; the pressure-correction solver was extended with a block correction, without which point SOR cannot conserve mass along a 400:1 artery.

- In Newtonian mode the solver reproduces the exact solution of its discrete equations to 2.6×10^{-6} (single-precision round-off) and the analytical Poiseuille gradient to 0.12%.
- With Bird–Carreau rheology the developed velocity, shear-rate and viscosity profiles agree with the exact solution within 0.2% and the pressure gradient within 0.07%. Shear-thinning blunts the profile (centreline velocity $1.44\bar{U}$ instead of $1.5\bar{U}$), develops over 50 mm ($14W$), and gives $\tau_w = 4.2$ Pa and a pressure drop of 335 Pa over the 140 mm segment.

References

S. Ali et al., Journal of X-Ray Science and Technology (2024) – LAD geometry and mean velocity (as cited in the original report).
Y.I. Cho, K.R. Kensey, Effects of the non-Newtonian viscosity of blood on flows in a diseased arterial vessel, *Biorheology* 28 (1991) 241–262.
B.P. Leonard, A stable and accurate convective modelling procedure based on quadratic upstream interpolation, *Comput. Methods Appl. Mech. Eng.* 19 (1979) 59–98.
A.J. Chorin, Numerical solution of the Navier–Stokes equations, *Math. Comp.* 22 (1968) 745–762.
F.H. Harlow, J.E. Welch, *Phys. Fluids* 8 (1965) 2182–2189 (MAC grid).
A. Settari, K. Aziz, A generalization of the additive correction methods for the iterative solution of matrix equations, *SIAM J. Numer. Anal.* 10 (1973) 506–521.
S.V. Patankar, *Numerical Heat Transfer and Fluid Flow*, Hemisphere, 1980.
NVIDIA, *CUDA C++ Programming Guide*.

Reproducing the results

`bash run.sh` builds `code/Blood_flow.cu`, runs the cases on the GPU if `nvcc` and an NVIDIA GPU are found, runs the same kernels by CPU emulation (about 15 min on one core) and calls `code/postprocess.py`. Single run: `nvcc -O3 -arch=native -o blood_flow code/Blood_flow.cu && ./blood_flow 2.0 15000 1.85 1000 1` (end time [s], VTK interval [steps], SOR ω, maximum sweeps, block correction on/off).

A Source code

```
// Blood through arteries (LAD) artery analysis
// The flow is assumed to be unsteady, incompressible, laminar, non-Newtonian, and fully developed
//
// Original author: Karthick J (OE23S018), OE5450 Numerical Techniques in Ocean Hydrodynamics, final project.
// Revision: corrections are marked [FIX], additions [ADDED], behaviour changes [CHANGED]. The kernel
// structure of the original (Initialize, ApplyBoundaryConditions, Shear_rate, X/YMomentum_solver,
// Divergence, PoissonSolverGS, CorrectVelocities, CalculateResidual, CalculateMaxVelocity, writeVTKFile)
// is kept. Method: explicit fractional-step (projection) method on a MAC staggered grid, QUICK
// convection, central diffusion with Bird-Carreau viscosity, red-black SOR pressure correction.
//
// Build (GPU): nvcc -O3 -arch=native -o blood_flow Blood_flow.cu
// Run: ./blood_flow [physical_time_s] [vtk_interval_steps] [sor_omega] [max_sor_sweeps] [block_correction 0/1]
// defaults: 5.0 1000 1.85 1000 1

#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <cuda.h>
#include <cuda_runtime.h>

// Defining the variables for the 2D flow simulation for host and device
```

```
#define rho 1.06e3f // density of blood in kg/m^3
#define L 0.14f // length of the artery in m
#define W 0.0035f // width of the artery in m
#define U 0.39f // velocity of the blood in m/s in x direction in the artery
                  // [FIX: U is the time-averaged MEAN velocity of the report (LAD data, 0.39 m/s);
                  // the inlet profile below is scaled so that its mean is U]
#define V 0.0f // velocity of the blood in m/s in y direction in the artery
#define DX 0.0001f // length of the cell in m for x direction (100micrometers)
#define DY 0.0001f // length of the cell in m for y direction (100micrometers)
#define dt 1.0e-5f // time step in seconds [FIX: no longer used inside the kernels, which now receive
                  // the adaptive time step actually taken (see main)]
#define NX ((int)(L/DX + 0.5f)) // number of cells in x direction [FIX: was (int)(L/DX)+1. The arrays
                               // hold CELLS, so +1 (a node count) gave 1401 x 36 cells = 140.1 x 3.6 mm;
                               // rounding instead of truncating also protects against L/DX = 1399.99..]
#define NY ((int)(W/DY + 0.5f)) // number of cells in y direction [FIX: 35 cells across W = 3.5 mm]
#define N (NX*NY) // total number of cells
#define del_P 2.8863e3f // initial pressure difference in Pa using the Poiseuille equation

// Bird-Carreau model parameters
#define mu0 0.056f // dynamic viscosity of blood in Pa.s at zero
#define mu_inf 0.0035f // dynamic viscosity of blood in Pa.s at infinite
#define lambda 3.13f // time constant in seconds
#define n 0.6f // flow index for shear thinning fluid
#define a 2.0f // bird-carreau model parameter

// Forward declarations
// [FIX: the original prototypes did not compile: parameters named dt, DX, DY, rho collide with the
// macros above, and two prototypes contained "int 1401, int 36". Parameters renamed dt_val, dx, dy,
// rho_val; nx, ny restored.]
__global__ void Initialize(float *u, float *v, float *p, float *visc, float *shear_rate, float p_init, int nx,
    int ny);
__global__ void ApplyBoundaryConditions(float *u, float *v, float *p, float p_out, int nx, int ny);
__global__ void Shear_rate(float *u, float *v, float *mu, float *shear_rate, int nx, int ny);
__global__ void Xmomentum_solver(float *u_x, float *u_y, float *p, float *mu, float *u_x_new, float dt_val,
    float dx, float dy, float rho_val, int nx, int ny);
__global__ void YMomentum_solver(float* u_x, float* u_y, float* p, float* v_new, float dt_val, float dx, float
    dy, float rho_val, float* mu, int nx, int ny);
__global__ void Divergence(float *u_x_new, float *v_new, float *divergence, int nx, int ny);
__global__ void PoissonSolverGS(float *divergence, float *p_prime, int nx, int ny, float omega, float dt_val,
    int color, float *residual);
__global__ void ColumnResidual(float *divergence, float *p_prime, int nx, int ny, float dt_val, float *col_res)
    ;
__global__ void BlockCorrectionTDMA(float *col_res, float *col_corr, float *work, int nx, int ny);
__global__ void ApplyBlockCorrection(float *p_prime, float *col_corr, int nx, int ny);
__global__ void CorrectVelocities(float *u, float *v, float *u_star, float *v_star, float *p, float *p_prime,
    float dt_val, int nx, int ny);
__global__ void CalculateResidual(float *d_u, float *d_u_old, float *d_v, float *d_v_old, float *residual, int
    nx, int ny);
__global__ void CalculateMaxVelocity(float *d_u, float *d_v, float *d_mu, float *max_velocity, float *max_u2_mu
    , float *max_inv_dt, int nx, int ny);
bool SolvePressurePoisson(float *d_divergence, float *d_p_prime, int nx, int ny, float omega, float tolerance,
                      float dt_val, int max_iter, int block_corr, float *d_residual, float *d_col_res,
                      float *d_col_corr, float *d_work, int *iters_done);
void writeVTKFile(const char* filename, float *u, float *v, float *p, float *visc, float *shear_rate, int nx,
    int ny, float dx, float dy);

// Bird-Carreau viscosity mu = mu_inf + (mu0 - mu_inf) [1 + (lambda*gamma)^a]^((n-1)/a)
// [ADDED as a function so that the kernels and the host-side wall-shear-stress output use one definition]
__host__ __device__ inline float carreau(float sr) {
   return mu_inf + (mu0 - mu_inf) * powf(1.0f + powf(lambda * sr, a), (n-1.0f)/a);
}

// QUICK face value: phi_f = 6/8 phi_U + 3/8 phi_D - 1/8 phi_UU (U = upstream, D = downstream,
// UU = far upstream). [ADDED helper; same formula as the original. FIX: falls back to first-order
// upwind (phi_f = phi_U) when UU lies outside the domain -- the original read u(i-2), u(j-2), u(j+2)
// at the first/last interior faces, i.e. outside the arrays.]
__device__ inline float quick(float phi_D, float phi_U, float phi_UU, bool has_UU) {
   return has_UU ? (0.375f*phi_D + 0.75f*phi_U - 0.125f*phi_UU) : phi_U;
}

// Initialize flow field with initial conditions
__global__ void Initialize(float *u, float *v, float *p, float *visc, float *shear_rate, float p_init, int nx,
    int ny) {
   int i = blockIdx.x * blockDim.x + threadIdx.x;
   int j = blockIdx.y * blockDim.y + threadIdx.y;
```

```
87
88      // Initialize pressure and viscosity at cell centers
89      if (i < nx && j < ny) {
90          int idx = i + j*nx; // Cell-centered index
91          p[idx] = p_init * (1.0f - (float)i/nx); // Linear pressure gradient
92          visc[idx] = mu0; // Initialize with zero-shear viscosity
93          shear_rate[idx] = 0.0f; // Initialize shear rate to zero
94      }
95
96      // Initialize x-velocity at east/west faces (staggered grid)
97      if (i <= nx && j < ny) { // Note: i can be nx for the last face
98          int idx_u = i + j*(nx+1); // Face-centered index for u
99
100         if (i == 0) {
101              u[idx_u] = U;
102         } else if (i == nx) {
103             // Outlet boundary condition
104             u[idx_u] = 0.0f; // [FIX: was u[idx_u-1], read while another thread initialises it (race)]
105         } else {
106             // Interior points
107             u[idx_u] = 0.0f; // Initialize to zero
108         }
109     }
110
111     // Initialize y-velocity at north/south faces (staggered grid)
112     if (i < nx && j <= ny) { // Note: j can be ny for the last face
113         int idx_v = i + j*nx; // Face-centered index for v
114         v[idx_v] = 0.0f; // Initialize to zero
115
116         // No-slip boundary conditions at walls (north/south)
117         if (j == 0 || j == ny) {
118             v[idx_v] = 0.0f;
119         }
120     }
121 }
122
123 // Apply boundary conditions for staggered grid
124 // Layout: p, mu at cell centres (i, j); u at the west face of cell i, u[i + j*(nx+1)], i = 0..nx;
125 // v at the south face of cell j, v[i + j*nx], j = 0..ny.
126 __global__ void ApplyBoundaryConditions(float *u, float *v, float *p, float p_out, int nx, int ny) {
127     int i = blockIdx.x * blockDim.x + threadIdx.x;
128     int j = blockIdx.y * blockDim.y + threadIdx.y;
129
130     // Inlet boundary (west face, x = 0)
131     if (i == 0 && j < ny) {
132         // u-velocity at inlet face (i=0)
133         // [FIX: u in row j sits at y = (j+0.5)DY, so the walls are at eta = -1, +1 only for
134         // eta = 2(j+0.5)/ny - 1 (the original 2j/(ny-1) - 1 put u = 0 in the first and last fluid
135         // rows), and the factor 1.5 makes the mean inflow equal to U (U(1-eta^2) has mean 2U/3)]
136         float y_norm = 2.0f * ((float)j + 0.5f)/ny - 1.0f;
137         u[j*(nx+1) + i] = 1.5f * U * (1.0f - y_norm*y_norm); // Parabolic profile
138
139         // v-velocity near inlet
140         v[j*nx + i] = 0.0f; // No y-component at inlet
141
142         // [FIX: removed p[inlet cell] = p[next cell]. Equal pressures on both sides of face 1 removed the
143         // pressure force on that face; the inlet pressure condition (zero normal gradient of the
144         // pressure correction, consistent with a prescribed inlet velocity) is applied in PoissonSolverGS]
145     }
146
147     // Outlet boundary (east face, x = nx)
148     if (i == nx && j < ny) {
149         // u-velocity at outlet face (i=nx)
150         u[j*(nx+1) + i] = u[j*(nx+1) + (i-1)]; // Zero gradient
151
152         // Pressure at outlet cells (i=nx-1)
153         if (i-1 >= 0) {
154             p[j*nx + (i-1)] = p_out; // Fixed pressure at outlet
155         }
156     }
157
158     // Wall boundaries (south face, y = 0)
159     if (j == 0 && i < nx) {
160         // v-velocity at bottom wall (j=0)
161         v[j*nx + i] = 0.0f; // No-slip condition
```

```
        // [FIX: removed u(row 0) = 0. On the staggered grid that row is at y = DY/2, inside the fluid;
        // forcing it to zero moved the wall one cell inwards (33 instead of 35 fluid rows). No-slip is
        // imposed with the ghost value u(-1) = -u(0) in the momentum and shear-rate kernels.]
    }

    // Wall boundaries (north face, y = ny)
    if (j == ny && i < nx) {
        // v-velocity at top wall (j=ny)
        v[j*nx + i] = 0.0f; // No-slip condition
        // [FIX: removed u(row ny-1) = 0 for the same reason (ghost value u(ny) = -u(ny-1))]
    }
}

// Calculate shear rate and update viscosity using Bird-Carreau model
__global__ void Shear_rate(float *u, float *v, float *mu, float *shear_rate, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // [FIX: evaluated in EVERY cell. The original skipped the wall-adjacent and inlet/outlet cells and
    // set mu = mu0 there, i.e. the highest viscosity exactly where the shear rate is largest.]
    if (i < nx && j < ny) {
        // Normal strain rates at the cell centre (exact on the staggered grid)
        float du_dx = (u[j*(nx+1) + (i+1)] - u[j*(nx+1) + i]) / DX;
        float dv_dy = (v[(j+1)*nx + i] - v[j*nx + i]) / DY;

        // [FIX: cross derivatives by central differences of cell-centred velocities. The original
        // (u_{j+1} - u_j)/DY (two-face average) is du/dy at y_{j+1/2}, half a cell above the centre,
        // and the same for dv/dx.] Ghost values: walls u(-1) = -u(0), u(ny) = -u(ny-1) (no slip);
        // inlet v = 0 (antisymmetric ghost); outlet zero gradient.
        float uc = 0.5f * (u[j*(nx+1) + i] + u[j*(nx+1) + (i+1)]);
        float ucN = (j < ny-1) ? 0.5f * (u[(j+1)*(nx+1) + i] + u[(j+1)*(nx+1) + (i+1)]) : -uc;
        float ucS = (j > 0) ? 0.5f * (u[(j-1)*(nx+1) + i] + u[(j-1)*(nx+1) + (i+1)]) : -uc;
        float vc = 0.5f * (v[j*nx + i] + v[(j+1)*nx + i]);
        float vcE = (i < nx-1) ? 0.5f * (v[j*nx + (i+1)] + v[(j+1)*nx + (i+1)]) : vc;
        float vcW = (i > 0) ? 0.5f * (v[j*nx + (i-1)] + v[(j+1)*nx + (i-1)]) : -vc;
        float du_dy = (ucN - ucS) / (2.0f * DY);
        float dv_dx = (vcE - vcW) / (2.0f * DX);

        // Calculate shear rate
        float sr = sqrtf(2.0f * (du_dx*du_dx + dv_dy*dv_dy) + (du_dy + dv_dx)*(du_dy + dv_dx));

        // Store at cell center
        int idx = j*nx + i;
        shear_rate[idx] = sr;

        // Update viscosity using Bird-Carreau model
        mu[idx] = carreau(sr);
    }
}

// X-momentum equation solver (explicit Euler predictor u*)
__global__ void Xmomentum_solver(float *u_x, float *u_y, float *p, float *mu, float *u_x_new, float dt_val,
     float dx, float dy, float rho_val, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if (i > nx || j >= ny) return;
    int idx_u = j*(nx+1) + i;

    // [FIX: the inlet (i = 0) and outlet (i = nx) faces are copied so that u* exists on every face;
    // the original left them uninitialised (cudaMalloc memory) and Divergence/CorrectVelocities read them]
    if (i == 0 || i == nx) { u_x_new[idx_u] = u_x[idx_u]; return; }

    // [FIX: solved on faces i = 1..nx-1 and ALL rows j = 0..ny-1. The original range i < nx-1,
    // 1 <= j < ny-1 never updated face nx-1 (so the outlet face stayed at u = 0, blocking the
    // channel) and froze the two wall-adjacent rows.]
    float uP = u_x[idx_u];
    float uE = u_x[idx_u + 1];
    float uW = u_x[idx_u - 1];
    float uN = (j < ny-1) ? u_x[idx_u + (nx+1)] : -uP; // ghost value: no-slip wall at y = W
    float uS = (j > 0) ? u_x[idx_u - (nx+1)] : -uP; // ghost value: no-slip wall at y = 0

    // 1. Calculate convection term using QUICK scheme
    // Calculate mass fluxes through cell faces (u-control volume centred on face i, between cells i-1 and i)
    float Fe = 0.5f * (uP + uE) * dy;
```

```
    float Fw = 0.5f * (uW + uP) * dy;

    // Interpolate v-velocities to u-cell faces
    // [FIX: two-point averages at the north and south faces of the u-CV. The original four-point
    // average is v at the CV centre (x_i, y_j) and was used for both faces.]
    float vn = 0.5f * (u_y[(j+1)*nx + i] + u_y[(j+1)*nx + (i-1)]);
    float vs = 0.5f * (u_y[j*nx + i] + u_y[j*nx + (i-1)]);

    float Fn = vn * dx;
    float Fs = vs * dx;

    // QUICK scheme for face values
    float phi_e, phi_w, phi_n, phi_s;
    if (Fe >= 0) phi_e = quick(uE, uP, uW, true);
    else phi_e = quick(uP, uE, (i+2 <= nx) ? u_x[idx_u + 2] : 0.0f, i+2 <= nx);
    if (Fw >= 0) phi_w = quick(uP, uW, (i-2 >= 0) ? u_x[idx_u - 2] : 0.0f, i-2 >= 0);
    else phi_w = quick(uW, uP, uE, true);
    if (Fn >= 0) phi_n = quick(uN, uP, uS, j-1 >= 0);
    else phi_n = quick(uP, uN, (j+2 <= ny-1) ? u_x[idx_u + 2*(nx+1)] : 0.0f, j+2 <= ny-1);
    if (Fs >= 0) phi_s = quick(uP, uS, (j-2 >= 0) ? u_x[idx_u - 2*(nx+1)] : 0.0f, j-2 >= 0);
    else phi_s = quick(uS, uP, uN, j+1 <= ny-1);

    float convection_term = -(Fe * phi_e - Fw * phi_w + Fn * phi_n - Fs * phi_s) / (dx * dy);

    // 2. Calculate diffusion term using central difference
    // [FIX: viscosity at the faces of the u-CV. East/west faces are the cell centres i and i-1; north/
    // south faces are cell corners (4-cell average, 2 cells at a wall). The original averaged
    // mu(i,j) with mu(i+1,j) and mu(i,j+1), i.e. values shifted by half a cell.]
    float mu_e = mu[j*nx + i];
    float mu_w = mu[j*nx + (i-1)];
    float mu_n = (j < ny-1) ? 0.25f * (mu[j*nx + i] + mu[j*nx + (i-1)] + mu[(j+1)*nx + i] + mu[(j+1)*nx + (i-1)
        ])
                            : 0.5f * (mu[j*nx + i] + mu[j*nx + (i-1)]);
    float mu_s = (j > 0) ? 0.25f * (mu[j*nx + i] + mu[j*nx + (i-1)] + mu[(j-1)*nx + i] + mu[(j-1)*nx + (i-1)])
                            : 0.5f * (mu[j*nx + i] + mu[j*nx + (i-1)]);

    // Calculate diffusion coefficients
    float De = mu_e * dy/dx;
    float Dw = mu_w * dy/dx;
    float Dn = mu_n * dx/dy;
    float Ds = mu_s * dx/dy;

    // Calculate diffusion term using coefficients
    float diffusion = De * (uE - uP) - Dw * (uP - uW) + Dn * (uN - uP) - Ds * (uP - uS);

    // 3. Calculate pressure gradient (staggered grid - pressure at cell centers)
    float pressure_gradient_term = (p[j*nx + i] - p[j*nx + (i-1)]) / dx;

    // 4. Update u_x using the momentum equation
    u_x_new[idx_u] = uP + dt_val * (convection_term + diffusion/(dx*dy*rho_val) - pressure_gradient_term/
        rho_val);
}

// Y-momentum equation solver (explicit Euler predictor v*)
__global__ void YMomentum_solver(float* u_x, float* u_y, float* p, float* v_new, float dt_val, float dx, float
     dy, float rho_val, float* mu, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if (i >= nx || j > ny) return;
    int idx = i + j*nx;

    // Faces that are not solved keep their boundary value: walls (j = 0, ny) and, as in the original,
    // the inlet and outlet cell columns (i = 0, nx-1) where v = 0 (developed in/outflow).
    // [FIX: they are copied into v* (the original left them uninitialised)]
    if (i < 1 || i > nx-2 || j < 1 || j > ny-1) { v_new[idx] = u_y[idx]; return; }
    // [FIX: solved for j = 1..ny-1; the original j < ny-1 froze the interior face j = ny-1 at v = 0]

    float vP = u_y[idx];
    float vE = u_y[idx + 1];
    float vW = u_y[idx - 1];
    float vN = u_y[idx + nx];
    float vS = u_y[idx - nx];

    // Interpolate u-velocities to v-cell faces (east and west)
```

```
    // [FIX: two-point averages at the east face (u at x_{i+1}, rows j-1 and j) and west face (x_i);
    // the original four-point average is u at the CV centre and was used for both faces]
    float ue = 0.5f * (u_x[(i+1) + j*(nx+1)] + u_x[(i+1) + (j-1)*(nx+1)]);
    float Fe = ue * dy;
    float uw = 0.5f * (u_x[i + j*(nx+1)] + u_x[i + (j-1)*(nx+1)]);
    float Fw = uw * dy;

    // Interpolate v-velocities to v-cell faces (north and south)
    float vn = 0.5f * (vP + vN);
    float Fn = vn * dx;
    float vs = 0.5f * (vS + vP);
    float Fs = vs * dx;

    // QUICK scheme for face values with flow direction check
    float phi_e, phi_w, phi_n, phi_s;
    if (Fe >= 0) phi_e = quick(vE, vP, vW, true);
    else phi_e = quick(vP, vE, (i+2 <= nx-1) ? u_y[idx + 2] : 0.0f, i+2 <= nx-1);
    if (Fw >= 0) phi_w = quick(vP, vW, (i-2 >= 0) ? u_y[idx - 2] : 0.0f, i-2 >= 0);
    else phi_w = quick(vW, vP, vE, true);
    if (Fn >= 0) phi_n = quick(vN, vP, vS, true);
    else phi_n = quick(vP, vN, (j+2 <= ny) ? u_y[idx + 2*nx] : 0.0f, j+2 <= ny);
    if (Fs >= 0) phi_s = quick(vP, vS, (j-2 >= 0) ? u_y[idx - 2*nx] : 0.0f, j-2 >= 0);
    else phi_s = quick(vS, vP, vN, true);

    // [FIX: viscosity at the faces of the v-CV: north/south faces are the cell centres (i,j) and
    // (i,j-1); east/west faces are cell corners (4-cell averages)]
    float mu_n = mu[i + j*nx];
    float mu_s = mu[i + (j-1)*nx];
    float mu_e = 0.25f * (mu[i + j*nx] + mu[(i+1) + j*nx] + mu[i + (j-1)*nx] + mu[(i+1) + (j-1)*nx]);
    float mu_w = 0.25f * (mu[i + j*nx] + mu[(i-1) + j*nx] + mu[i + (j-1)*nx] + mu[(i-1) + (j-1)*nx]);

    // Calculate diffusion coefficients
    float De = mu_e * dy/dx;
    float Dw = mu_w * dy/dx;
    float Dn = mu_n * dx/dy;
    float Ds = mu_s * dx/dy;

    // Calculate convection term
    float convection_term = -(Fe * phi_e - Fw * phi_w + Fn * phi_n - Fs * phi_s) / (dx * dy);

    // Calculate diffusion term using coefficients
    float diffusion = De * (vE - vP) - Dw * (vP - vW) + Dn * (vN - vP) - Ds * (vP - vS);

    // Pressure gradient for v-velocity at (i,j-1/2)
    float pressure_gradient_term = (p[i + j*nx] - p[i + (j-1)*nx]) / dy;

    // Update v-velocity
    v_new[idx] = vP + dt_val * (convection_term + diffusion/(dx*dy*rho_val) - pressure_gradient_term/rho_val);
}

// Calculate divergence for pressure correction
__global__ void Divergence(float *u_x_new, float *v_new, float *divergence, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // [FIX: every cell (the original skipped the boundary cells, whose divergence stayed uninitialised
    // and whose mass balance was never enforced)]
    if (i < nx && j < ny) {
        float du_dx = (u_x_new[j*(nx+1) + (i+1)] - u_x_new[j*(nx+1) + i]) / DX;
        float dv_dy = (v_new[(j+1)*nx + i] - v_new[j*nx + i]) / DY;
        divergence[j*nx + i] = du_dx + dv_dy;
    }
}

// Pressure-correction equation lap(p') = (rho/dt) div(u*) by red-black Gauss-Seidel with SOR.
// Unknowns: cells i = 0..nx-2 (the last column holds the outlet pressure, p' = 0). A neighbour
// coefficient is zero where the corresponding face velocity is not corrected: inlet face (u prescribed),
// walls (v = 0) and the v-faces of the inlet column (v = 0, see YMomentum_solver).
__global__ void PoissonSolverGS(float *divergence, float *p_prime, int nx, int ny, float omega, float dt_val,
    int color, float *residual) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // [FIX: red-black ordering. The original updated every cell in one launch while reading neighbours
    // from the array being written (race: a non-deterministic mix of Jacobi and Gauss-Seidel), and it
```

```
    // never updated the boundary cells, so no pressure boundary condition entered the solve.]
    if (i >= nx-1 || j >= ny || ((i + j) & 1) != color) return;
    int idx = j*nx + i;

    float aE = 1.0f/(DX*DX);
    float aW = (i > 0) ? 1.0f/(DX*DX) : 0.0f;
    float aN = (i > 0 && j < ny-1) ? 1.0f/(DY*DY) : 0.0f;
    float aS = (i > 0 && j > 0) ? 1.0f/(DY*DY) : 0.0f;
    float sum = aE * p_prime[idx + 1];
    if (aW > 0.0f) sum += aW * p_prime[idx - 1];
    if (aN > 0.0f) sum += aN * p_prime[idx + nx];
    if (aS > 0.0f) sum += aS * p_prime[idx - nx];

    // [FIX: source sign and time step. u = u* - (dt/rho) grad p' with div u = 0 requires
    // lap p' = +(rho/dt) div u*; the original used -rho*div/dt with the constant macro dt = 1e-5 s
    // instead of the adaptive time step actually taken.]
    float p_old = p_prime[idx];
    float p_gs = (sum - rho * divergence[idx] / dt_val) / (aE + aW + aN + aS);

    // Apply SOR (Successive Over-Relaxation)
    float p_new = (1.0f - omega) * p_old + omega * p_gs;
    p_prime[idx] = p_new;

    // Calculate local residual for convergence check
    atomicAdd(residual, fabsf(p_new - p_old));
}

// [ADDED] Block (additive) correction along the artery (Settari & Aziz 1973; Patankar 1980, Sec. 7.2).
// Point SOR on a 1400 x 35 domain reduces the error component that is uniform across the artery
// (a change of the axial flow rate) by only ~1e-5 per sweep, so the original solver could not make
// the cross-sectional flow rate equal to the inflow. The correction adds c_i to every cell of column i,
// with c_i chosen so that the column-summed residual vanishes (exact mass balance of every cross-section).
__global__ void ColumnResidual(float *divergence, float *p_prime, int nx, int ny, float dt_val, float *col_res)
     {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= nx-1) return;
    float R = 0.0f;
    for (int j = 0; j < ny; j++) {
        int idx = j*nx + i;
        float pP = p_prime[idx];
        float r = rho * divergence[idx] / dt_val - (p_prime[idx + 1] - pP) / (DX*DX);
        if (i > 0) {
            r -= (p_prime[idx - 1] - pP) / (DX*DX);
            if (j < ny-1) r -= (p_prime[idx + nx] - pP) / (DY*DY);
            if (j > 0) r -= (p_prime[idx - nx] - pP) / (DY*DY);
        }
        R += r; // residual of sum_nb a_nb (p'_nb - p'_P) = (rho/dt) div
    }
    col_res[i] = R;
}

// Single-thread Thomas algorithm for A_W c_{i-1} - (A_W + A_E) c_i + A_E c_{i+1} = R_i,
// i = 0..nx-2, A_E = A_W = ny/DX^2 (A_W = 0 at the inlet), c_{nx-1} = 0 (outlet column).
__global__ void BlockCorrectionTDMA(float *col_res, float *col_corr, float *work, int nx, int ny) {
    if (blockIdx.x != 0 || threadIdx.x != 0) return;
    int m = nx - 1;
    float A = (float)ny / (DX*DX);
    // forward elimination (work = modified super-diagonal)
    float b0 = -A; // i = 0: A_W = 0
    work[0] = A / b0;
    col_corr[0] = col_res[0] / b0;
    for (int i = 1; i < m; i++) {
        float bi = -2.0f*A;
        float den = bi - A * work[i-1];
        work[i] = (i < m-1) ? A / den : 0.0f; // c_{nx-1} = 0
        col_corr[i] = (col_res[i] - A * col_corr[i-1]) / den;
    }
    for (int i = m-2; i >= 0; i--) col_corr[i] -= work[i] * col_corr[i+1];
}

__global__ void ApplyBlockCorrection(float *p_prime, float *col_corr, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    if (i < nx-1 && j < ny) p_prime[j*nx + i] += col_corr[i];
}
```

```
456
457 // Host-side wrapper for the red-black SOR iteration with convergence check
458 bool SolvePressurePoisson(float *d_divergence, float *d_p_prime, int nx, int ny, float omega, float tolerance,
459                           float dt_val, int max_iter, int block_corr, float *d_residual, float *d_col_res,
460                           float *d_col_corr, float *d_work, int *iters_done) {
461     dim3 block(16, 16);
462     dim3 grid((nx + block.x - 1) / block.x, (ny + block.y - 1) / block.y);
463     dim3 block1(128);
464     dim3 grid1((nx + block1.x - 1) / block1.x);
465     float h_residual;
466
467     // [FIX: incremental formulation. The momentum predictor already contains grad p^n, so the Poisson
468     // equation is solved for the correction p' (zero initial guess, i.e. warm start p^{n+1} = p^n);
469     // the original solved for the full pressure and then applied grad p a second time.]
470     cudaMemset(d_p_prime, 0, nx * ny * sizeof(float));
471
472     for (int iter = 0; iter < max_iter; ++iter) {
473         if (block_corr && iter % 10 == 0) { // [ADDED] block correction every 10 sweeps
474             ColumnResidual<<<grid1, block1>>>(d_divergence, d_p_prime, nx, ny, dt_val, d_col_res);
475             BlockCorrectionTDMA<<<1, 1>>>(d_col_res, d_col_corr, d_work, nx, ny);
476             ApplyBlockCorrection<<<grid, block>>>(d_p_prime, d_col_corr, nx, ny);
477         }
478         // Reset residual
479         cudaMemset(d_residual, 0, sizeof(float));
480
481         // Perform one red-black SOR sweep (two half-sweeps)
482         PoissonSolverGS<<<grid, block>>>(d_divergence, d_p_prime, nx, ny, omega, dt_val, 0, d_residual);
483         PoissonSolverGS<<<grid, block>>>(d_divergence, d_p_prime, nx, ny, omega, dt_val, 1, d_residual);
484
485         // Check convergence (mean |change| per sweep)
486         cudaMemcpy(&h_residual, d_residual, sizeof(float), cudaMemcpyDeviceToHost);
487         h_residual /= (nx * ny); // Normalize by grid size
488
489         if (h_residual < tolerance) {
490             *iters_done = iter + 1;
491             if (block_corr) {
492                 ColumnResidual<<<grid1, block1>>>(d_divergence, d_p_prime, nx, ny, dt_val, d_col_res);
493                 BlockCorrectionTDMA<<<1, 1>>>(d_col_res, d_col_corr, d_work, nx, ny);
494                 ApplyBlockCorrection<<<grid, block>>>(d_p_prime, d_col_corr, nx, ny);
495             }
496             return true; // Converged
497         }
498     }
499     *iters_done = max_iter;
500     if (block_corr) { // [ADDED] final block correction: exact column mass balance on exit
501         ColumnResidual<<<grid1, block1>>>(d_divergence, d_p_prime, nx, ny, dt_val, d_col_res);
502         BlockCorrectionTDMA<<<1, 1>>>(d_col_res, d_col_corr, d_work, nx, ny);
503         ApplyBlockCorrection<<<grid, block>>>(d_p_prime, d_col_corr, nx, ny);
504     }
505     return false; // Did not converge within max_iter
506 }
507
508 // Correct velocities using the pressure correction, and update the pressure
509 __global__ void CorrectVelocities(float *u, float *v, float *u_star, float *v_star, float *p, float *p_prime,
     float dt_val, int nx, int ny) {
510     int i = blockIdx.x * blockDim.x + threadIdx.x;
511     int j = blockIdx.y * blockDim.y + threadIdx.y;
512
513     // d-coefficient of the projection step [FIX: adaptive dt (was the constant macro dt)]
514     float d_c = dt_val / rho;
515
516     // Correct u-velocity (staggered, west face of cell i)
517     // [FIX: all interior faces i = 1..nx-1 and all rows (was 1 <= j < ny-1); boundary faces take u*]
518     if (i <= nx && j < ny) {
519         int idx_u = j*(nx+1) + i;
520         if (i >= 1 && i <= nx-1) {
521             float dp_dx = (p_prime[j*nx + i] - p_prime[j*nx + (i-1)]) / DX; // [FIX: gradient of p', not p]
522             u[idx_u] = u_star[idx_u] - d_c * dp_dx;
523         } else {
524             u[idx_u] = u_star[idx_u];
525         }
526     }
527
528     // Correct v-velocity (staggered, south face of cell j) on the faces solved in YMomentum_solver
529     if (i < nx && j <= ny) {
```

```
        int idx_v = j*nx + i;
        if (i >= 1 && i <= nx-2 && j >= 1 && j <= ny-1) {
            float dp_dy = (p_prime[j*nx + i] - p_prime[(j-1)*nx + i]) / DY;
            v[idx_v] = v_star[idx_v] - d_c * dp_dy;
        } else {
            v[idx_v] = v_star[idx_v];
        }
    }

    // [ADDED] pressure update p^{n+1} = p^n + p' (the original never updated p from the correction)
    if (i < nx && j < ny) p[j*nx + i] += p_prime[j*nx + i];
}

// Calculate residual between time steps (sum of squared changes)
__global__ void CalculateResidual(float *d_u, float *d_u_old, float *d_v, float *d_v_old, float *residual, int
     nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // [FIX: all (nx+1) x ny u-faces and nx x (ny+1) v-faces. Inside "if (i < nx && j < ny)" the tests
    // i < nx+1 and j < ny+1 were always true, so the outlet faces and the top-wall row were skipped.]
    if (i <= nx && j < ny) {
        float u_diff = d_u[j*(nx+1) + i] - d_u_old[j*(nx+1) + i];
        atomicAdd(residual, u_diff*u_diff);
    }
    if (i < nx && j <= ny) {
        float v_diff = d_v[j*nx + i] - d_v_old[j*nx + i];
        atomicAdd(residual, v_diff*v_diff);
    }
}

// Calculate maximum velocity (CFL), and the two explicit-stability indicators [ADDED]:
// max_inv_dt = max over cells of |u|/DX + |v|/DY + 2(mu/rho)(1/DX^2 + 1/DY^2) (high-wavenumber bound)
// max_u2_mu = max over cells of |u|^2/mu (low-wavenumber bound)
__global__ void CalculateMaxVelocity(float *d_u, float *d_v, float *d_mu, float *max_velocity, float *max_u2_mu
     ,
                                     float *max_inv_dt, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if (i < nx && j < ny) {
        // Calculate velocity magnitude at cell center
        float u_center = 0.5f * (d_u[j*(nx+1) + i] + d_u[j*(nx+1) + (i+1)]);
        float v_center = 0.5f * (d_v[j*nx + i] + d_v[(j+1)*nx + i]);
        float vel2 = u_center*u_center + v_center*v_center;
        float velocity_mag = sqrtf(vel2);
        float mu_c = d_mu[j*nx + i];
        float inv_dt = fabsf(u_center)/DX + fabsf(v_center)/DY + 2.0f*(mu_c/rho)*(1.0f/(DX*DX) + 1.0f/(DY*DY));

        // Update maximum velocity using atomic operation (valid for non-negative floats)
        atomicMax((int*)max_velocity, __float_as_int(velocity_mag));
        atomicMax((int*)max_u2_mu, __float_as_int(vel2 / mu_c));
        atomicMax((int*)max_inv_dt, __float_as_int(inv_dt));
    }
}

// Write VTK file for visualization
void writeVTKFile(const char* filename, float *u, float *v, float *p, float *visc, float *shear_rate, int nx,
     int ny, float dx, float dy) {
    FILE *fp = fopen(filename, "w");
    if (fp == NULL) {
        printf("Error opening file for VTK output\n");
        return;
    }

    // VTK header
    fprintf(fp, "# vtk DataFile Version 3.0\n");
    fprintf(fp, "Blood Flow Simulation\n");
    fprintf(fp, "ASCII\n");
    fprintf(fp, "DATASET RECTILINEAR_GRID\n");
    // [FIX: DIMENSIONS counts grid POINTS: (nx+1) x (ny+1) for nx x ny cells. The original wrote
    // nx x ny points and (nx-1)(ny-1) cells, dropping the last cell column and row.]
    fprintf(fp, "DIMENSIONS %d %d 1\n", nx+1, ny+1);

    // Coordinate information
```

```
    fprintf(fp, "X_COORDINATES %d float\n", nx+1);
    for (int i = 0; i <= nx; i++) {
        fprintf(fp, "%f ", i*dx);
    }
    fprintf(fp, "\n");

    fprintf(fp, "Y_COORDINATES %d float\n", ny+1);
    for (int j = 0; j <= ny; j++) {
        fprintf(fp, "%f ", j*dy);
    }
    fprintf(fp, "\n");

    fprintf(fp, "Z_COORDINATES 1 float\n");
    fprintf(fp, "0.0\n");

    // Cell-centered data
    fprintf(fp, "CELL_DATA %d\n", nx*ny);

    // Pressure field
    fprintf(fp, "SCALARS pressure float 1\n");
    fprintf(fp, "LOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++) {
        for (int i = 0; i < nx; i++) {
            fprintf(fp, "%f\n", p[j*nx + i]);
        }
    }

    // Velocity magnitude
    fprintf(fp, "SCALARS velocity_magnitude float 1\n");
    fprintf(fp, "LOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++) {
        for (int i = 0; i < nx; i++) {
            // Interpolate u and v to cell centers
            float u_center = 0.5f * (u[j*(nx+1) + i] + u[j*(nx+1) + (i+1)]);
            float v_center = 0.5f * (v[j*nx + i] + v[(j+1)*nx + i]);
            float vel_mag = sqrtf(u_center*u_center + v_center*v_center);
            fprintf(fp, "%f\n", vel_mag);
        }
    }

    // [ADDED] viscosity and shear rate (the host copies were made but never written)
    fprintf(fp, "SCALARS viscosity float 1\n");
    fprintf(fp, "LOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++) for (int i = 0; i < nx; i++) fprintf(fp, "%e\n", visc[j*nx + i]);
    fprintf(fp, "SCALARS shear_rate float 1\n");
    fprintf(fp, "LOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++) for (int i = 0; i < nx; i++) fprintf(fp, "%e\n", shear_rate[j*nx + i]);

    // Velocity vectors
    fprintf(fp, "VECTORS velocity float\n");
    for (int j = 0; j < ny; j++) {
        for (int i = 0; i < nx; i++) {
            // Interpolate u and v to cell centers
            float u_center = 0.5f * (u[j*(nx+1) + i] + u[j*(nx+1) + (i+1)]);
            float v_center = 0.5f * (v[j*nx + i] + v[(j+1)*nx + i]);
            fprintf(fp, "%f %f 0.0\n", u_center, v_center);
        }
    }

    fclose(fp);
    printf("Saved VTK file: %s\n", filename);
}

// [ADDED] cell-centred final fields (x y u v p mu gamma) and wall shear stress for post-processing
void writeFinalFields(float *u, float *v, float *p, float *visc, float *shear_rate, int nx, int ny) {
    FILE *fp = fopen("final_fields.dat", "w");
    fprintf(fp, "# x[m] y[m] u[m/s] v[m/s] p[Pa] mu[Pa.s] gamma_dot[1/s] (cell centres, %d x %d)\n", nx, ny);
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++) {
            float uc = 0.5f * (u[j*(nx+1) + i] + u[j*(nx+1) + (i+1)]);
            float vc = 0.5f * (v[j*nx + i] + v[(j+1)*nx + i]);
            fprintf(fp, "%.7e %.7e %.7e %.7e %.7e %.7e %.7e\n", (i + 0.5f)*DX, (j + 0.5f)*DY, uc, vc,
                    p[j*nx + i], visc[j*nx + i], shear_rate[j*nx + i]);
        }
    fclose(fp);
```

```
    // wall shear stress: wall gradient implied by the no-slip ghost value u(-1) = -u(0), i.e. 2 u_0/DY with
    // u_0 the cell-centre velocity at DY/2 (the wall flux of the finite-volume scheme; exact for the discrete
    // Poiseuille solution); tau_w = mu(gamma_w) * gamma_w
    fp = fopen("wall_shear_stress.csv", "w");
    fprintf(fp, "x_m,tau_bottom_Pa,tau_top_Pa\n");
    for (int i = 0; i < nx; i++) {
        float ub0 = 0.5f * (u[i] + u[i+1]);
        float ut0 = 0.5f * (u[(ny-1)*(nx+1) + i] + u[(ny-1)*(nx+1) + i+1]);
        float gb = 2.0f * ub0 / DY, gt = 2.0f * ut0 / DY;
        fprintf(fp, "%.7e,%.7e,%.7e\n", (i + 0.5f)*DX, carreau(fabsf(gb))*gb, carreau(fabsf(gt))*gt);
    }
    fclose(fp);
}

int main(int argc, char **argv) {
    // Initialize variables and parameters
    float *h_u, *h_v, *h_p, *h_visc, *h_shear_rate, *h_div;
    float *d_u, *d_v, *d_p, *d_visc, *d_shear_rate;
    float *d_u_star, *d_v_star, *d_p_prime, *d_divergence;
    float *d_u_old, *d_v_old;
    float *d_residual, *d_max_velocity, *d_max_u2_mu, *d_max_inv_dt, *d_col_res, *d_col_corr, *d_work; // [
        ADDED]

    // Timing variables
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    // Allocate host memory
    h_u = (float*)malloc(sizeof(float) * (NX+1) * NY);
    h_v = (float*)malloc(sizeof(float) * NX * (NY+1));
    h_p = (float*)malloc(sizeof(float) * NX * NY);
    h_visc = (float*)malloc(sizeof(float) * NX * NY);
    h_shear_rate = (float*)malloc(sizeof(float) * NX * NY);
    h_div = (float*)malloc(sizeof(float) * NX * NY);

    // Allocate device memory
    cudaMalloc((void**)&d_u, sizeof(float) * (NX+1) * NY);
    cudaMalloc((void**)&d_v, sizeof(float) * NX * (NY+1));
    cudaMalloc((void**)&d_p, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_visc, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_shear_rate, sizeof(float) * NX * NY);

    // Allocate memory for intermediate values
    cudaMalloc((void**)&d_u_star, sizeof(float) * (NX+1) * NY);
    cudaMalloc((void**)&d_v_star, sizeof(float) * NX * (NY+1));
    cudaMalloc((void**)&d_p_prime, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_divergence, sizeof(float) * NX * NY);

    // Allocate memory for old values (needed for temporal discretization)
    cudaMalloc((void**)&d_u_old, sizeof(float) * (NX+1) * NY);
    cudaMalloc((void**)&d_v_old, sizeof(float) * NX * (NY+1));

    // [FIX: reduction scalars and solver work arrays allocated once (the original called cudaMalloc/
    // cudaFree inside the time loop and inside every pressure solve)]
    cudaMalloc((void**)&d_residual, sizeof(float));
    cudaMalloc((void**)&d_max_velocity, sizeof(float));
    cudaMalloc((void**)&d_max_u2_mu, sizeof(float));
    cudaMalloc((void**)&d_max_inv_dt, sizeof(float));
    cudaMalloc((void**)&d_col_res, sizeof(float) * NX);
    cudaMalloc((void**)&d_col_corr, sizeof(float) * NX);
    cudaMalloc((void**)&d_work, sizeof(float) * NX);

    // Set execution configuration
    // [FIX: the launch grid must cover i = 0..NX and j = 0..NY (outlet u-faces, top-wall v-faces);
    // sizing it with NX, NY only works by accident when NX, NY are not multiples of 16]
    dim3 blockSize(16, 16);
    dim3 gridSize((NX + 1 + blockSize.x - 1) / blockSize.x,
                  (NY + 1 + blockSize.y - 1) / blockSize.y);

    // Initialize flow field on device
    Initialize<<<gridSize, blockSize>>>(d_u, d_v, d_p, d_visc, d_shear_rate, del_P, NX, NY);
    cudaDeviceSynchronize();

    // Simulation parameters
```

```
751     double physicalTime = 5.0; // Total physical time to simulate (seconds)
752     double currentTime = 0.0; // Current physical time [CHANGED: double, so that ~1e5 additions of
753                               // dt ~ 4e-5 s do not accumulate float round-off]
754     int timeStep = 0; // Current time step
755     float cfl = 0.5f; // Target CFL number
756     float maxResidual = 1.0f; // Initial residual
757     float convergenceTolerance = 1e-5f; // Convergence criteria
758     int maxIterations = 1; // [FIX: was 1000 "SIMPLE iterations". Each pass of that loop
759                               // re-predicted from the CURRENT u (not u^n) with the full dt, i.e.
760                               // advanced the flow by one more explicit step, while the clock was
761                               // advanced only once: up to 1000 dt per reported dt. One predictor-
762                               // projection per time step is the explicit fractional-step method.]
763     int outputInterval = 1000; // VTK output every N time steps
764     float omega = 1.85f; // [CHANGED: SOR factor was 1.5 (inside the loop). After the block
765                               // correction the slowest error modes vary across the 35-cell width;
766                               // omega_opt = 2/(1+sin(pi/35)) ~ 1.84]
767     int maxPoissonIter = 1000; // Maximum SOR sweeps per pressure solve (as in the original)
768     int blockCorrection = 1; // [ADDED] 1 = block correction on (0 reproduces point SOR only)
769     int logInterval = 100; // [ADDED] history (CSV) every N steps
770
771     // [CHANGED: parameters from the command line instead of scanf, so that runs can be scripted]
772     if (argc > 1) physicalTime = atof(argv[1]);
773     if (argc > 2) outputInterval = atoi(argv[2]);
774     if (argc > 3) omega = atof(argv[3]);
775     if (argc > 4) maxPoissonIter = atoi(argv[4]);
776     if (argc > 5) blockCorrection = atoi(argv[5]);
777     printf("Grid %d x %d cells (%.4f m x %.4f m), t_end = %.4f s, VTK every %d steps, SOR omega = %.3f, "
778            "max sweeps = %d, block correction = %d\n", NX, NY, NX*DX, NY*DY, physicalTime, outputInterval,
779            omega, maxPoissonIter, blockCorrection);
780
781     // Calculate adaptive time step based on CFL condition
782     // [FIX: the original used only the CFL step 0.5*DX/U = 1.28e-4 s. Explicit Euler for the momentum
783     // equation is stable only if, cell by cell,
784     // dt * ( |u|/DX + |v|/DY + 2 nu (1/DX^2 + 1/DY^2) ) <= 1 (high wavenumbers: diffusion+convection)
785     // dt * |u|^2 / (2 nu) <= 1 (low wavenumbers, FTCS-type)
786     // With mu = mu0 = 0.056 Pa s the first bound is 3.7e-5 s; 1.28e-4 s is 3.5 times too large. The
787     // pure diffusion limit rho DX^2/(4 mu0) = 4.7e-5 s is not enough either: at that value the
788     // (upwind-biased) QUICK dissipation pushes the cell-to-cell oscillation mode to |G| = 1 + CFL > 1.]
789     float maxVelocity = 1.5f * U; // Initial estimate (peak of the inlet profile)
790     float safety = 0.9f; // [ADDED]
791     float adaptiveDt = safety / (maxVelocity/DX + 2.0f*(mu0/rho)*(1.0f/(DX*DX) + 1.0f/(DY*DY)));
792     adaptiveDt = fminf(adaptiveDt, cfl * DX / maxVelocity);
793     printf("Initial time step: %e seconds\n", adaptiveDt);
794
795     // Create output file
796     FILE *outfile = fopen("blood_flow_results.csv", "w");
797     fprintf(outfile, "Time,TimeStep,PoissonSweeps,RMS_dudt,MaxVelocity,MaxPressure,dt,MaxDivRel,Qout_over_Qin\n
          ");
798
799     // Start timing
800     cudaEventRecord(start, 0);
801     int poissonIters = 0;
802     float h_max_velocity = maxVelocity, h_max_u2_mu = 0.0f, h_max_inv_dt = 0.0f;
803
804     // Time advancement loop
805     while (currentTime < physicalTime - 1e-9) {
806         timeStep++;
807
808         // Store previous time step values for temporal discretization
809         cudaMemcpy(d_u_old, d_u, sizeof(float) * (NX+1) * NY, cudaMemcpyDeviceToDevice);
810         cudaMemcpy(d_v_old, d_v, sizeof(float) * NX * (NY+1), cudaMemcpyDeviceToDevice);
811
812         // Predictor-projection (one pass per time step, see maxIterations)
813         int iterations = 0;
814         maxResidual = 1.0f;
815
816         while (iterations < maxIterations && maxResidual > convergenceTolerance) {
817             // Step 1: Apply boundary conditions
818             ApplyBoundaryConditions<<<gridSize, blockSize>>>(d_u, d_v, d_p, 0.0f, NX, NY);
819
820             // Step 2: Momentum prediction
821             Xmomentum_solver<<<gridSize, blockSize>>>(d_u, d_v, d_p, d_visc, d_u_star, adaptiveDt, DX, DY, rho,
                    NX, NY);
822             YMomentum_solver<<<gridSize, blockSize>>>(d_u, d_v, d_p, d_v_star, adaptiveDt, DX, DY, rho, d_visc,
                    NX, NY);
```

```
823
824             // Step 3: Calculate divergence
825             Divergence<<<gridSize, blockSize>>>(d_u_star, d_v_star, d_divergence, NX, NY);
826
827             // Step 4: Solve pressure correction equation
828             SolvePressurePoisson(d_divergence, d_p_prime, NX, NY, omega, convergenceTolerance/10.0f, adaptiveDt,
829                                  maxPoissonIter, blockCorrection, d_residual, d_col_res, d_col_corr, d_work, &
830                                      poissonIters);
831
832             // Step 5: Correct velocities and pressure
833             CorrectVelocities<<<gridSize, blockSize>>>(d_u, d_v, d_u_star, d_v_star, d_p, d_p_prime, adaptiveDt,
834                  NX, NY);
835             ApplyBoundaryConditions<<<gridSize, blockSize>>>(d_u, d_v, d_p, 0.0f, NX, NY); // [ADDED] outlet
836                 face
837
838             // Step 6: Update viscosity using Bird-Carreau model
839             Shear_rate<<<gridSize, blockSize>>>(d_u, d_v, d_visc, d_shear_rate, NX, NY);
840
841             // Velocity change over the step [CHANGED: reported as RMS du/dt]
842             cudaMemset(d_residual, 0, sizeof(float));
843             CalculateResidual<<<gridSize, blockSize>>>(d_u, d_u_old, d_v, d_v_old, d_residual, NX, NY);
844             float h_res;
845             cudaMemcpy(&h_res, d_residual, sizeof(float), cudaMemcpyDeviceToHost);
846             maxResidual = sqrtf(h_res / ((NX+1)*NY + NX*(NY+1))) / adaptiveDt;
847             iterations++;
848         }
849
850         // Update current time
851         currentTime += adaptiveDt;
852         float dtTaken = adaptiveDt; // [ADDED] for the history file
853
854         // Calculate maximum velocity and the stability indicators for adaptive time stepping
855         cudaMemset(d_max_velocity, 0, sizeof(float));
856         cudaMemset(d_max_u2_mu, 0, sizeof(float));
857         cudaMemset(d_max_inv_dt, 0, sizeof(float));
858         CalculateMaxVelocity<<<gridSize, blockSize>>>(d_u, d_v, d_visc, d_max_velocity, d_max_u2_mu,
859              d_max_inv_dt, NX, NY);
860         cudaMemcpy(&h_max_velocity, d_max_velocity, sizeof(float), cudaMemcpyDeviceToHost);
861         cudaMemcpy(&h_max_u2_mu, d_max_u2_mu, sizeof(float), cudaMemcpyDeviceToHost);
862         cudaMemcpy(&h_max_inv_dt, d_max_inv_dt, sizeof(float), cudaMemcpyDeviceToHost);
863
864         // Update time step based on CFL condition
865         // [FIX: dt = min(CFL, high-wavenumber, low-wavenumber) limits, evaluated cell by cell (see above)]
866         float dtNew = safety / h_max_inv_dt;
867         if (h_max_velocity > 0.01f) { // Avoid division by very small values
868             dtNew = fminf(dtNew, cfl * DX / h_max_velocity);
869         }
870         if (h_max_u2_mu > 0.0f) dtNew = fminf(dtNew, safety * 2.0f / (rho * h_max_u2_mu));
871
872         // Limit time step growth for stability
873         adaptiveDt = fminf(dtNew, 1.5f * adaptiveDt); // [FIX: was fminf(adaptiveDt, 1.5f*adaptiveDt), a no-op]
874
875         // Adjust final time step to exactly reach physicalTime
876         // [FIX: the remainder is split over the last two steps. A last step of arbitrary size (possibly
877         // ~1e-9 s) turns the small residual divergence into a pressure correction ~ rho*div*DX^2/dt that
878         // corrupts the final pressure field.]
879         double remaining = physicalTime - currentTime;
880         if (remaining < 2.0 * adaptiveDt) {
881             adaptiveDt = (float)((remaining > adaptiveDt) ? 0.5 * remaining : remaining);
882         }
883
884         bool lastStep = (currentTime >= physicalTime - 1e-9);
885         // [ADDED] history every logInterval steps: pressure maximum, mass conservation
886         if (timeStep % logInterval == 0 || timeStep % outputInterval == 0 || lastStep) {
887             cudaMemcpy(h_u, d_u, sizeof(float) * (NX+1) * NY, cudaMemcpyDeviceToHost);
888             cudaMemcpy(h_p, d_p, sizeof(float) * NX * NY, cudaMemcpyDeviceToHost);
889             Divergence<<<gridSize, blockSize>>>(d_u, d_v, d_divergence, NX, NY);
890             cudaMemcpy(h_div, d_divergence, sizeof(float) * NX * NY, cudaMemcpyDeviceToHost);
891
892             // Calculate maximum pressure
893             float maxPressure = 0.0f, maxDiv = 0.0f, qin = 0.0f, qout = 0.0f;
894             for (int k = 0; k < NX*NY; k++) {
895                 if (fabs(h_p[k]) > maxPressure) maxPressure = fabs(h_p[k]);
896                 if ((k % NX) < NX-1 && fabs(h_div[k]) > maxDiv) maxDiv = fabs(h_div[k]);
897             }
```

```
            for (int j = 0; j < NY; j++) { qin += h_u[j*(NX+1)] * DY; qout += h_u[j*(NX+1) + NX] * DY; }

            // Write to output file
            fprintf(outfile, "%.6f,%d,%d,%.6e,%.6f,%.6f,%.6e,%.6e,%.6f\n",
                   currentTime, timeStep, poissonIters, maxResidual, h_max_velocity, maxPressure, dtTaken,
                   maxDiv * DX / U, qout / qin);

            // Output results at specified intervals
            if (timeStep % outputInterval == 0 || lastStep) {
                printf("Time: %.4f s, Step: %d, Poisson sweeps: %d, RMS du/dt: %.6e, Max Velocity: %.4f m/s, "
                       "max|div|DX/U: %.2e, Qout/Qin: %.6f\n", currentTime, timeStep, poissonIters, maxResidual,
                       h_max_velocity, maxDiv * DX / U, qout / qin);

                // Copy data back to host for output
                cudaMemcpy(h_v, d_v, sizeof(float) * NX * (NY+1), cudaMemcpyDeviceToHost);
                cudaMemcpy(h_visc, d_visc, sizeof(float) * NX * NY, cudaMemcpyDeviceToHost);
                cudaMemcpy(h_shear_rate, d_shear_rate, sizeof(float) * NX * NY, cudaMemcpyDeviceToHost);

                // Write field data to VTK file for visualization
                char filename[256];
                sprintf(filename, "blood_flow_t%.4f.vtk", currentTime);
                writeVTKFile(filename, h_u, h_v, h_p, h_visc, h_shear_rate, NX, NY, DX, DY);
            }
        }
    }

    // Stop timing and report
    cudaEventRecord(stop, 0);
    cudaEventSynchronize(stop);
    float elapsedTime;
    cudaEventElapsedTime(&elapsedTime, start, stop);
    cudaError_t err = cudaGetLastError(); // [ADDED]
    if (err != cudaSuccess) printf("CUDA error: %s\n", cudaGetErrorString(err));
    printf("Total simulation time: %.2f seconds (%d time steps, %.3f ms per step)\n",
           elapsedTime/1000.0f, timeStep, elapsedTime/timeStep);

    // [ADDED] final cell-centred fields and wall shear stress
    writeFinalFields(h_u, h_v, h_p, h_visc, h_shear_rate, NX, NY);

    // Close output file
    fclose(outfile);

    // Clean up
    cudaEventDestroy(start);
    cudaEventDestroy(stop);

    // Free host memory
    free(h_u);
    free(h_v);
    free(h_p);
    free(h_visc);
    free(h_shear_rate);
    free(h_div);

    // Free device memory
    cudaFree(d_u);
    cudaFree(d_v);
    cudaFree(d_p);
    cudaFree(d_visc);
    cudaFree(d_shear_rate);
    cudaFree(d_u_star);
    cudaFree(d_v_star);
    cudaFree(d_p_prime);
    cudaFree(d_divergence);
    cudaFree(d_u_old);
    cudaFree(d_v_old);
    cudaFree(d_residual);
    cudaFree(d_max_velocity);
    cudaFree(d_max_u2_mu);
    cudaFree(d_max_inv_dt);
    cudaFree(d_col_res);
    cudaFree(d_col_corr);
    cudaFree(d_work);

    return 0;
```

```
969 }
```